Tarea 2: Control y Visualización en una Pantalla

Por Ricardo José Colom Palero

En esta presentación, se plantearán las bases para el desarrollo y realización de la siguiente práctica de la asignatura, en la que se abordará el diseño de un generador de sincronismo para gestionar la visualización en una pantalla.

Previamente a la realización de los diseños propuestos en la tarea debe realizarse un estudio de la información facilitada en la Lessons.

Project 2: Control and Display on a Screen

By Ricardo José Colom Palero

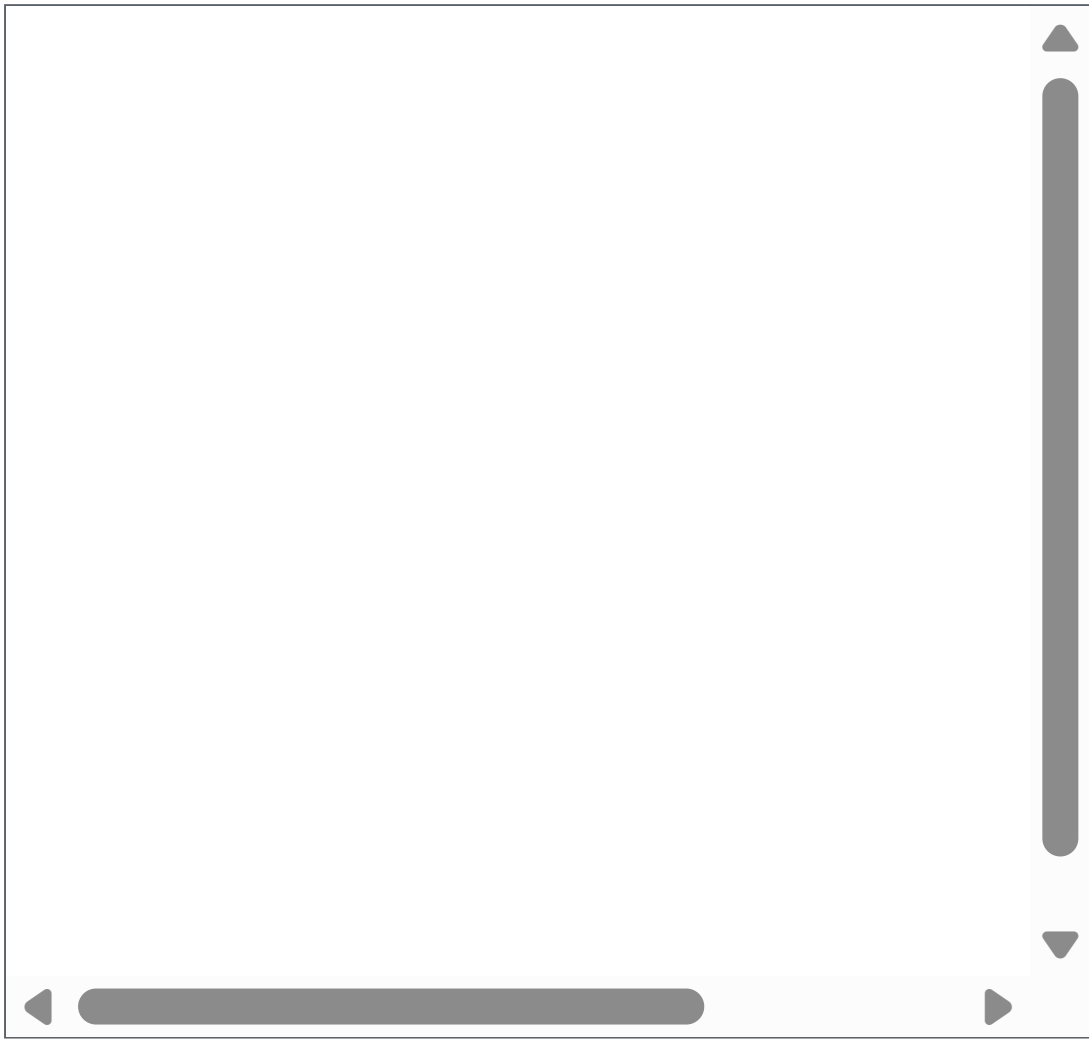
In this lesson we set the foundations for the development of Project #2, in which we will approach the design of a synchronism generator to control the display on a screen.

Before attempting the implementation of the proposed designs in this task, one should review the information in this lesson.

Objetivos

Goals

ABRIR EN UNA VENTANA NUEVA



Como objetivos de la práctica se plantean el aprendizaje de las características y modos de funcionamiento de una pantalla, ya que es la aplicación sobre la que se focaliza el diseño. En primer lugar será necesario realizar el desarrollo e implementación de un elemento de control que genere las líneas de sincronismo que se requieren para la visualización en una pantalla. Para abordar con éxito el diseño habrá que profundizar en los conceptos del lenguaje Verilog HDL, orientados al uso en el diseño de elementos de control. Muy importante en el desarrollo de esta práctica es la metodología de verificación funcional de diseños. Como elementos complementarios a la visualización, se abordará el uso de memorias ROM, así como el manejo y funcionamiento de un teclado.

The goal of this project is to learn the characteristics and operation modes of a display screen, as this is the application which the design focuses on. First, it will be necessary to develop and implement a control element that generates the synchronism signals required for visualization on a screen. In order to tackle the design successfully, we will take a closer look at the elements of the Verilog HDL oriented to the design of control elements. The methodology for functional verification of designs will be very important in this project. As accessory elements to the display on a screen, we will also need to review the use of ROM memories, as well as the operation and control of a keyboard.

Visualización en una Pantalla

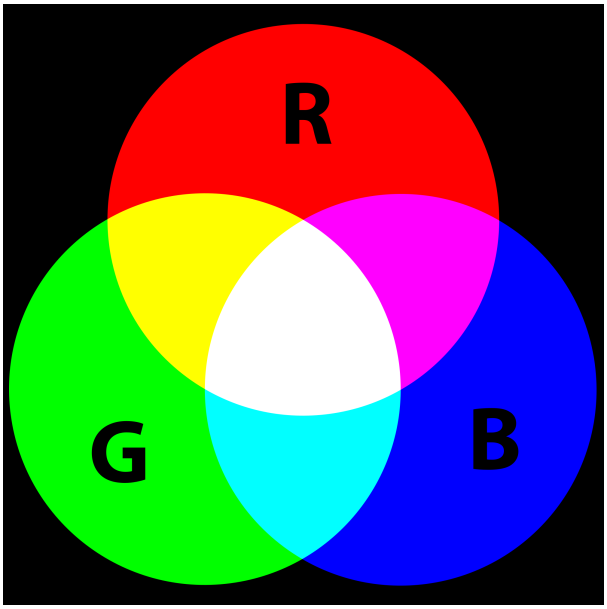
Display on a Screen

En el arte de la pintura, el diseño gráfico, el diseño visual, la fotografía, la imprenta y en la televisión, la teoría del color es un grupo de reglas básicas sobre la mezcla de colores para conseguir el efecto deseado combinando colores de luz o pigmento. La luz blanca se puede producir combinando el rojo, el verde y el azul, mientras que combinando pigmentos cian, magenta y amarillo se produce el color negro.

Un sistema de color aditivo implica que se emita luz directamente de una fuente de iluminación de algún tipo. El proceso de reproducción aditiva normalmente utiliza luz roja, verde y azul para producir el resto de colores. Combinando los tres colores primarios de luz con las mismas intensidades, se produce el blanco. Variando la intensidad de cada luz de cada color se puede ver el espectro completo de colores.

In the art of painting, graphic design, visual design, photography, press and TV, the theory of color is a basic set of rules about the mixing of colors in order to obtain the desired effect by combining lights or pigments of different colors. White light can be produced by combining red, green and blue light, while black paint can be obtained by combining cyan, magenta and yellow pigment.

An additive color system implies the direct emission of light from some sort of lighting source. The additive production process typically uses red, green and blue lights to generate the rest of colors. Combining those three primary light colors with the same intensity results in white light. Changing the light intensity of each color one may generate the whole color spectrum.



El RGB es un modelo de color basado en la síntesis aditiva, con él que es posible representar un color mediante la mezcla por adición de los tres colores de luz primarios, que son rojo, verde y azul (RGB, iniciales en inglés de los colores primarios). Todos los colores posibles que pueden ser creados por la mezcla de estas tres luces de color, forman el espectro de color. Cuando ningún color de luz está presente, se percibe el color negro. Los colores primarios de luz tienen aplicación en los monitores de un ordenador, televisores, proyectores de vídeo y todos aquellos sistemas que utilizan combinaciones de materiales que fosforecen en el rojo, verde y azul.

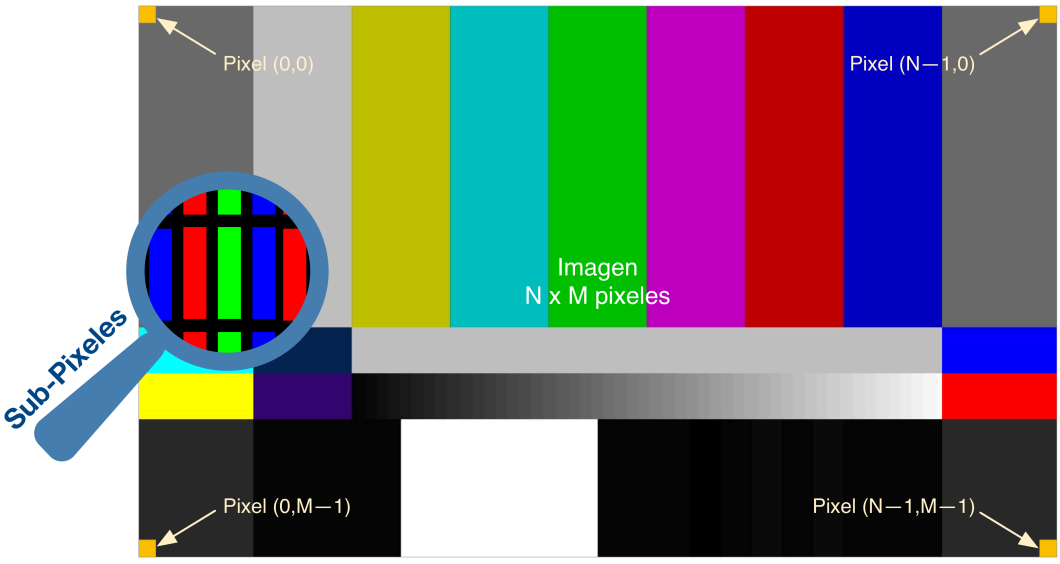
The RGB model is a color model based on additive synthesis, under which it is possible to represent a color as an additive mixture of the three primary light colors (Red, Green and Blue). All possible colors that can be created by combining those three make up the color spectrum. When no light color is present, one perceives the color black. Primary light colors find their applications in screens and display units for computers, TV, video projectors, and in general all systems that use combinations of materials that phosphoresce in the red, green and blue spectrum.

Representación de Imágenes

Representation of Images

Una imagen se define formalmente como la distribución espacial de la luz irradiada en un plano. Matemáticamente, una imagen es una función continua de dos variables, que son las coordenada **x** e **y** de un punto del espacio bidimensional y que representa la intensidad de luz irradiada en ese punto. Las imágenes son continuas por lo que se refiere a los valores de las coordenadas y a la intensidad. Para representar una imagen de manera digital es necesario discretizar las coordenadas y la intensidad, así una imagen digital es aquella que los valores de las coordenadas y de la intensidad son finitos y discretos. A cada punto en el plano con coordenadas discretas se le denomina pixel. Por tanto, una imagen digital se representa como conjunto de puntos distribuidos en el plano, formando una rejilla rectangular de **N** franjas verticales o columnas y **M** franjas horizontales o filas, lo que se conoce como una imagen de **N** por **M** píxeles.

An image can be formally defined as the spatial distribution of irradiated light on a plane. Mathematically, an image is a continuous function in two variables: the **x** and **y** coordinates of a point in bidimensional space, and it represents the light intensity at this point. In images, the values of coordinates and intensity are continuous. In order to represent an image digitally it becomes necessary to discretize the coordinates and the intensity. A digital image is thus an image in which the values of the coordinates and intensities are finite and discrete. Each point in the plane with discrete coordinates is called a **pixel** (for ***pic**ture **el**ement*). A digital image is therefore represented as a set of points distributed on a plane, forming a rectangular matrix with **N** vertical strips or columns and **M** horizontal strips or rows. This is called an image of **N × M** pixels.



Es adecuado realizar un inciso, para comentar que matemáticamente una matriz de puntos se nombra como número de filas por número de columnas, en este ejemplo correspondería decir **M** por **N** pixeles, sin embargo, cuando se trata de pantallas siempre se indica en primer lugar la dimensión horizontal y por tanto el número de columnas y por ello se ha dicho que la imagen es de **N** por **M** pixeles.

Por tanto, una imagen digital es una estructura que representa una rejilla rectangular de N por M píxeles o puntos de color, denominada matriz, que se puede visualizar en un monitor, papel u otro dispositivo de representación. Se considera el primer pixel él que ocupa el margen superior izquierdo que corresponde con la posición o coordenada **(0, 0)**.

Cada punto o pixel de la imagen debe contener información de color, presentada en canales separados o subpixeles que representan los componentes primarios del color que se pretende mostrar, en cualquier modelo de color, bien sea RGB, o cualquier otro disponible.

La profundidad de color o bits por pixel (bpp) es un concepto de la computación gráfica que se refiere a la cantidad de bits de información necesarios para representar el color de un píxel en una imagen digital. Debido a la naturaleza del sistema binario de numeración, una profundidad de **n** bits implica que cada píxel de la imagen puede tener 2^n posibles valores y por lo tanto, representar 2^n colores distintos. Debido a la aceptación

It is convenient to make a remark here: the mathematical convention for notation of a matrix is as number of rows times number of columns, so in this example one would have an **M × N** matrix of pixels. However, when referring to display screens we usually follow a different convention and states the horizontal dimension first, i.e. the number of columns, so we will speak of an **N × M** pixel image instead.

To recap, a digital image is a structure that represents a rectangular lattice of N × M pixels (or color points), called a matrix, that may be displayed on screen, paper, or any other means of representation. The pixel on the top left corner, corresponding to the position or coordinate **(0,0)**, is considered to be the first pixel in the image.

Each point or pixel in the image must contain color information, encoded in separate channels or sub-pixels that represent the primary components of the color to be displayed in some color model (be it RGB or any other available model).

The color depth, measured in bits per pixel (bpp), is a concept in graphic computation that refers to the number of bits needed to represent the color of each pixel in a digital image. Due to the nature of the binary number system, a depth of n bits implies that each pixel can take 2^n possible values and therefore represent 2^n different colors. Since 8-bit bytes are almost universally accepted as basic units of information in storage devices, color depth values tend to be divisors or

prácticamente universal de los octetos de 8 bits como unidades básicas de información en los dispositivos de almacenamiento, los valores de profundidad de color suelen ser divisores o múltiplos de 8, a saber 1, 2, 4, 8, 16, 24 y 32, con la excepción de la profundidad de color de 10 o 15, usada por ciertos dispositivos gráficos.

multiples of 8, typically 1, 2, 4, 8, 16, 24 or 32, with the exception of color depth 10 or 15 which are used by some graphics devices.

Imagen en Pantalla

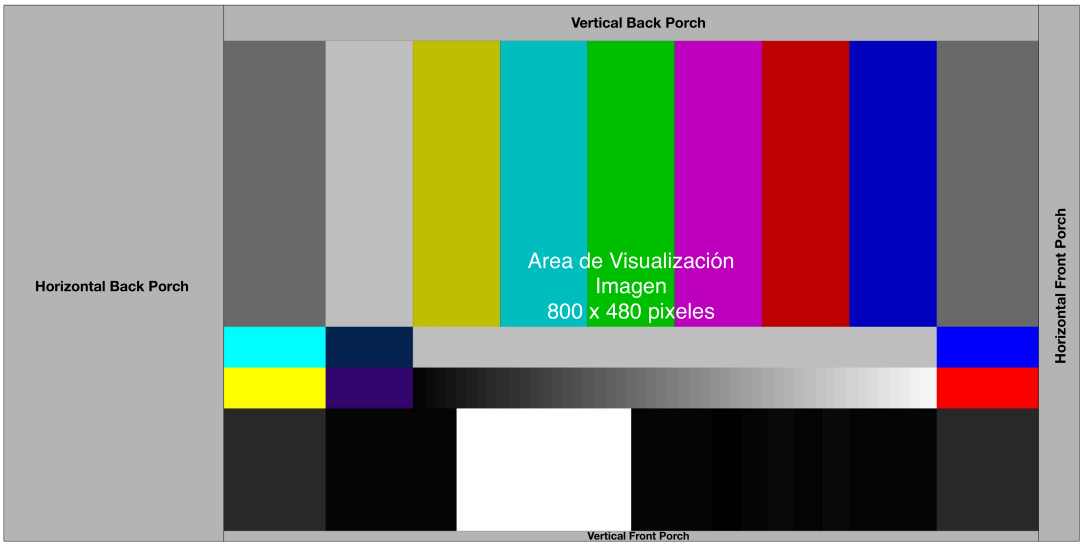
Onscreen Image

A la hora de visualizar una imagen digital en una pantalla hay que tener presente las características de la pantalla, fundamentalmente las dimensiones del área de visualización y la profundidad del color. Habitualmente una de las características principales de una pantalla es el tamaño de la imagen, indicando en primer lugar la dimensión horizontal en número de pixeles y en segundo lugar la dimensión vertical. El tamaño de la imagen que facilita un fabricante de pantallas se refiere al área de visualización de la pantalla, es decir cuantos pixeles se van a ver en la pantalla.

Sin embargo, a nivel de funcionamiento interno una pantalla tiene un área mucho mayor que viene caracterizada por unas zonas "**negras**" (pintadas de gris en la imagen), que se encuentran alrededor de la zona de visualización, haciendo que el área de trabajo interna de la electrónica de control o los márgenes de funcionamiento sea mucho mayores. Estas zonas se utilizan para estabilizar las señales de sincronismo que permiten marcar las pautas de una correcta visualización de la imagen. Estas zonas "**negras**" se conocen por sus propios nombres: *Horizontal Back Porch* y *Front Porch* para las zonas de la izquierda y de la derecha del área de visualización respectivamente y *Vertical Back Porch* y *Front Porch* para la zona superior e inferior al área de visualización respectivamente.

In order to display a digital image on a screen one must keep in mind the characteristics of the screen, mainly the dimensions of the display area and the color depth. One of the main features of a screen is usually the image size, indicating firstly the horizontal dimension and secondly the vertical dimension, in number of pixels. The image size given by a vendor refers to the display area of the screen, that is, how many pixels can be displayed on the screen.

However, considering its internal operation, a screen has a much bigger area that is characterized by "**black**" zones (shown as gray in the image below) around the display zone, making the internal working area for the control electronics and the operating margins much larger. These zones are used to stabilize the synchronism signals used to set the tone for correct image display. These "**black**" zones have specific names: *Horizontal Back Porch* and *Front Porch* for the zones to the left and right of the display area, respectively, and *Vertical Back Porch* and *Front Porch* for the zones above and below the display area, respectively.



Cabe destacar que las zonas alrededor del área de visualización no son simétricas o de tamaño igual, las denominadas "*Back Porch*" son mayores que sus correspondientes "*Front Porch*" ya que las señales de sincronismo requieren de un mayor tiempo de estabilización antes de la visualización.

Por tanto, a la hora de realizar un elemento para controlar la visualización en una pantalla, habrá que tener en cuenta que dicha pantalla tiene unas zonas "**negras**" alrededor del área de visualización de la imagen.

These zones around the display area are neither symmetrical nor have equal sizes. The *Back Porch* areas are larger than their *Front Porch* counterparts because synchronism signals require a longer time to stabilize before display.

As a consequence, when designing a circuit for controlling display on a screen, one must consider the existence of these "**black**" areas around the image display area.

Funcionamiento de la Visualización

How Image Display Works

La técnica que utilizan las pantallas habitualmente para la representación de imágenes, suele ser la misma que utilizaban en el pasado los Tubos de Rayos Catódicos, los cuales empleaban un haz de electrones que iba recorriendo una superficie fosforescente para "dibujar" en ella la imagen que se pretendía representar. Gracias a la fosforescencia de la superficie y a la persistencia del ojo humano parecían imágenes estables, pero realmente el haz de electrones estaba continuamente moviéndose por la superficie dibujando imágenes. El haz de electrones tardaba aproximadamente 20 ms en recorrer toda la pantalla, dibujando una imagen o "*frame*".

En esta ocasión se trata de pantallas planas, en las que no hay un haz de electrones y lo que se pretende representar es una imagen digital constituida por una matriz de pixeles, pero la técnica sigue siendo la misma: realizar un barrido de la pantalla mediante un punto denominado "*dot clock*". El "*dot clock*" comenzará su barrido en el extremo superior izquierda de la pantalla, fuera de la zona de visualización de la imagen, en la zona correspondiente a los "*porch*", e irá desplazándose hacia la derecha de pixel en pixel a cada golpe de flanco de reloj. Como los cambios de un pixel al adyacente se producen en función de una señal de reloj, que hay que proporcionar a la pantalla, es por lo que se denomina "*dot clock*" o punto de reloj. Cuando el "*dot clock*" llega al borde derecho de la pantalla, hay que indicarle que ha llegado al final de la fila y que por tanto debe ir al principio de la siguiente fila, lo cual se

The technique used by most screen for image display is usually the same as was used in the past for Cathode Ray Tubes, which are based on an electron beam that scans a phosphorescent screen in order to "draw" the image to be represented on it. The images seem to be stable thanks to the phosphorescence of the display surface and the persistence of the human eye, but the electron beam is in fact continuously moving across the screen drawing images. The electron beam takes approximately 20 ms to scan the whole screen, displaying an image or "*frame*".

Nowadays we typically use flat-panel screens that feature no such electron beam and display digital images composed of a pixel matrix, but the technique is still the same: scanning the screen using a pointer called "*dot clock*". The dot clock starts scanning at the top left corner of the screen, in the zone corresponding to the "back porch" areas outside of the image display area, and then moves to the right pixel by pixel with each clock cycle. Changes from one pixel to the next happen according to a clock signal that must be supplied to the screen, hence the name "dot clock". When the dot clock arrives at the right edge of the screen, a signal must be issued to indicate that it has reached the end of the current row and must therefore move to the start of the next one. This is accomplished with a *horizontal sync(hronism)* signal, that consists in a negative pulse issued whenever the dot clock is located at the last pixel in a row.

realiza con una señal denominada sincronismo horizontal, que consiste en un pulso negativo cuando el "*dot clock*" se encuentra en el último pixel de la fila.

Una vez el "*dot clock*" se encuentre en el último pixel de la última fila, hay que indicarle que ha recorrido toda la pantalla y por tanto debe volver al principio o a la primera fila de pixeles, de esta labor se encarga una señal denominada sincronismo vertical, que consiste en un pulso negativo que indica que una nueva imagen o "*frame*" debe comenzar.

Once the dot clock reaches the last pixel in the last row, another signal must be issued to indicate that the whole screen has been scanned and it must therefore go back to the first row. A *vertical sync(chronism)* signal is used for that purpose, consisting in a negative pulse indicating the start of a new image or *frame*.

Hay que pensar que el "*dot clock*" es un punto de la pantalla que debe ser guiado externamente para recorrer toda la pantalla, por ello se utiliza una señal de reloj, que le va indicando que avance un pixel en cada flanco, una señal de sincronismo horizontal que le indica que ha llegado al borde derecho y tiene que volver al principio de la siguiente fila y una señal de sincronismo vertical que le indica que ha llegado al final de las filas y debe volver a la primera fila o al principio de la pantalla.

El hecho que el "*dot clock*" deba moverse rápidamente desde el lado derecho al izquierdo para comenzar a barrer una nueva fila o de abajo a arriba para una nueva imagen, justifica la existencia de esos márgenes que se han denominado "*porch*" alrededor de el área de visualización de la imagen, sobre todo teniendo en cuenta que la técnica proviene de los Tubos de Rayos Catódicos, en los que el funcionamiento se basaba en un haz de electrones con un control analógico para realizar el barrido a lo largo de la pantalla.

En paralelo a todo ello, hay tres señales digitales denominadas RGB que representan la intensidad de cada color primario, para cada pixel de la pantalla. Estas señales deben tomar un valor efectivo cuando el "*dot clock*" se encuentra dentro de la zona de visualización de la pantalla, y ese valor corresponderá con el color que se deba representar en el pixel en el que se encuentre el "*dot clock*", para formar la imagen. Por ello, las señales RGB debe ir cambiando síncrona-

It is convenient to think of the dot clock as a dot on the screen that must be externally guided in order to scan the whole screen. Thus, we use a clock signal that makes it advance one pixel on each clock edge, a horizontal sync signal that tells it to go back to the start of the next row when it reaches the right edge, and a vertical sync signal that tells it to go back to the start of the screen once it reaches the end.

Note that the dot clock must move quickly from the right to the left side in order to start scanning a new row, or from the bottom to the top in order to start a new frame. This fact justifies the existence of the aforementioned margins (the "porches") around the image display area, especially if we keep in mind that this technique stems from the Cathode Ray Tube days, whose operation was based on an analogically controlled electron beam performing the screen scan.

In parallel with this, there are three digital signals called RGB that represent the intensity of each primary color for each pixel on the screen. In order to compose an image, these signals must take an effective value when the dot clock is located within the image display area; this value will be the color represented at the pixel where the dot clock is located. Therefore, the RGB signals must change synchronously with the movement of the dot clock in order to represent the desired image. This implies that changes in the RGB signals will be timed with the same clock signal used to move the dot clock.

mente con el movimiento del "*dot clock*" para poder representar la imagen deseada, esto implica que el cambio de las señales RGB se realizará con el mismo reloj que utiliza para hacer que el "*dot clock*" se mueva.

ABRIR EN UNA VENTANA NUEVA

En la animación se representa una emulación del funcionamiento del barrido de la pantalla, se muestra como el "*dot clock*" va moviéndose por la pantalla, como ve van activando los pulsos de sincronismo horizontal y vertical. Por supuesto la emulación se realiza a una velocidad mucho más lenta de lo habitual, para poder ver el punto, ya que se supone que para ver una imagen estable en pantalla, el punto debería tardar 20 ms en recorrer toda la pantalla.

Importante destacar dos aspectos, en primero es que el punto representado cambia de color cuando entra en la zona del área de visualización para demostrar que en ese momento tienen protagonismo las señales RGB y la otra consideración es que para no hacerlo demasiado largo, en un momento determinado el punto tomara un atajo directo para ir al final de la pantalla.

The animation above represents an emulation of the screen scan operation, showing how the dot clock moves across the screen and the horizontal and vertical sync pulses are generated. Of course, this animation at a much slower pace than usual in order to be able to see the movement of the dot clock, as it usually takes 20 ms to scan the whole screen in order to display stable images.

Let us remark two aspects of this animation. First, the dot clock changes its color when it enters the display area in order to show when the RGB signals are used. Second, the dot fast-forwards through the display area in order to shorten the animation length.

Pantalla Táctil LTM

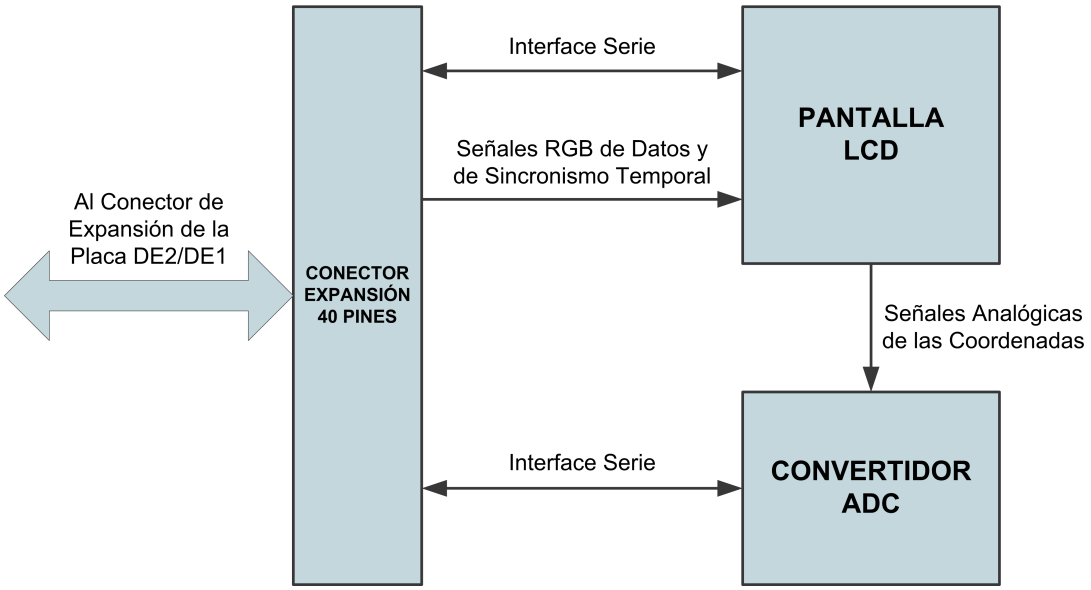
LTM Touchscreen

La pantalla táctil que se utilizará en el desarrollo de esta práctica es la fabricada por TerAsic, para las placas del programa universitario de Altera, como la DE2. Se trata de una pantalla de 4,3 pulgadas con una relación de aspecto de 15/9, que tiene una resolución de 800 por 480 pixeles de 24 bits, pudiendo representar hasta 16 millones de colores. La pantalla dispone de un convertidor analógico digital, que captura el punto de contacto.

In this project we will use the touchscreen developed by TerAsic for the Altera University Program evaluation boards such as the DE2. It is a 4.3 inch screen with a 15:9 aspect ratio and a resolution of 800 x 480 pixels with 24 bpp, and is able to represent up to 16 million colors. The screen also features an analog-to-digital converter used to capture the contact point location.

Estructura Pantalla Táctil LTM

Structure of the LTM Touchscreen



En esta pantalla, la gestión de la visualización y la captura del punto de contacto son totalmente independientes, esto permite utilizar únicamente la visualización sin tener que capturar punto de contacto, hecho que se va a utilizar en esta práctica. Además la parte de visualización lleva un interface serie, que permite ajustar la configuración y los parámetros de la visualización en la pantalla, tales como la resolución o los ajustes de color. Sin embargo, la visualización se puede realizar utilizando los valores de ajuste por defecto de la pantalla, así que no se será necesario realizar ningún ajuste adicional y por tanto no se utilizará el interface serie. En definitiva la práctica se centrará en las señales RGB de datos y de sincronismo temporal para la visualización en la pantalla.

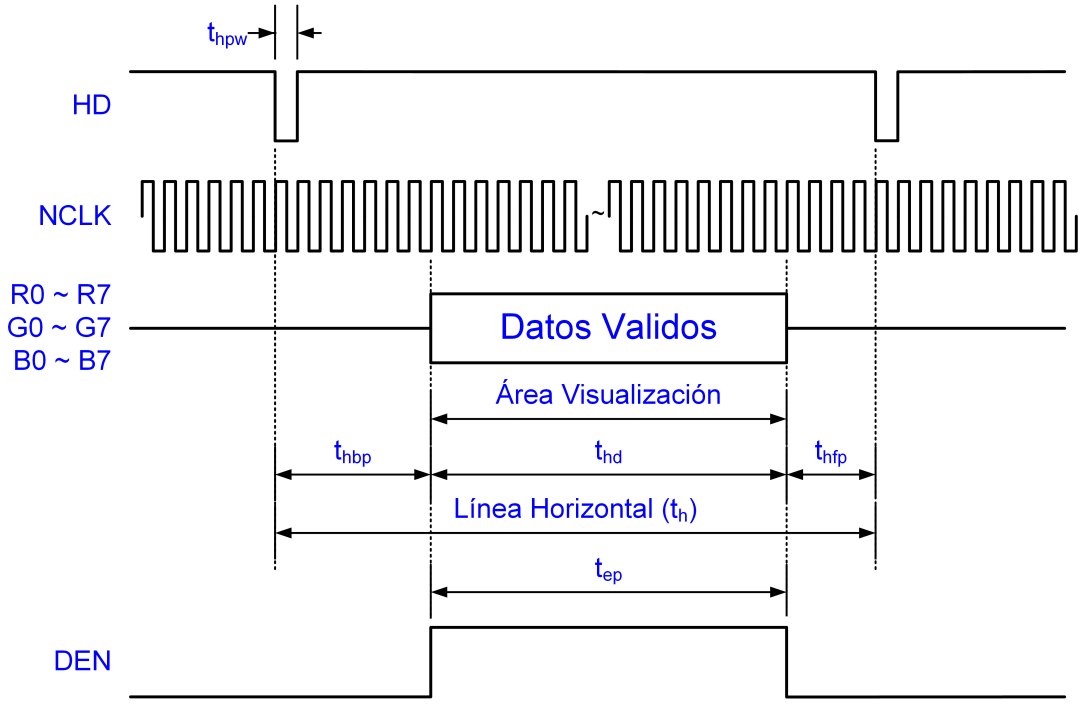
For this screen, display and capture of the contact point are managed in a completely independent way. This will allow us to focus on the display in this Project, without needing to worry about touch recognition. Besides, the display circuitry includes a serial interface used to adjust the value of several screen display parameters, such as resolution or color settings. Nevertheless, it is possible to display images using the default screen settings with no need for additional adjustments, therefore it will not be necessary to use the serial interface. Instead, the Project will focus on the RGB data and synchronism signals for the display.

Diagrama Temporización Horizontal

En la Figura se muestran los requerimientos temporales básicos para la visualización de cada fila (horizontal) que se quiere mostrar en la pantalla.

Horizontal Timing Diagram

The Figure below shows the basic timing specifications for the display of each horizontal row to be represented on screen.



Un pulso activo a nivel bajo de duración concreta (t_{hpw}), se utiliza como señal de sincronismo horizontal (HD), para indicar el final de una línea y el comienzo de una nueva línea de datos. Después del pulso de sincronismo horizontal, las señales de datos (RGB), todavía no son válidas durante un periodo de tiempo conocido como horizontal "*back porch*" (t_{hbp}). Tras este tiempo, vendrá el tiempo de visualización de los datos en pantalla (t_{hd}), en el que los datos se aplican a los pixeles, mediante un recorrido horizontal por la fila seleccionada. Durante el tiempo de visualización de los datos en la pantalla (t_{hd}), la señal de habilitación de la pantalla (DEN) debe permanecer a nivel alto. Finalmente hay un nuevo periodo temporal denominado horizontal "*front porch*" (t_{hfp}), en el que las señales de datos (RGB) vuelven a no ser válidos, y que tras concluir se produce un nuevo pulso de sincronismo horizontal.

Es importante destacar, que todas las señales funcionan síncronamente con una señal de reloj denominada NCLK, que va marcando los cambios de las señales con el flanco positivo de dicha señal, la pantalla no deja de ser un elemento pasivo, por tanto la señal de reloj le sirve como referencia para marcar el ritmo de los datos RGB y del cambio de las señales de sincronismo.

An active low pulse of fixed length (t_{hpw}) is used as the horizontal sync signal (**HD**) in order to determine the end of a row and the beginning of a new row. After the horizontal sync pulse, the data signals (RGB) are invalid for a period of time known as the horizontal "back porch" (t_{hbp}). After this we have the time for onscreen data display (t_{hd}), when the data are applied to the pixels by way of a horizontal scan across the selected row. During this data display time (t_{hd}), the Display Enable signal (**DEN**) must stay high. Finally, there is another time interval called horizontal "front porch" (t_{hfp}) when the RGB data signals are again invalid. After this time interval, the new horizontal sync pulse is generated.

It is important to notice that all of these signals are synchronous with a clock signal called **NCLK**, whose positive edges time changes in the control signals. The screen is a passive device, and so this clock signal is used as a reference to time the RGB data and synchronism signals.

Parámetro	Símbolo	Valor	Unidad
Frecuencia de reloj	NCLK	33,2	MHz
Datos Horizontales	t_{hd}	800	NCLK
Total Línea Horizontal	t_{h}	1056	NCLK
Ancho de Pulso HS	t_{hpw}	1	NCLK
Horizontal "Back Porch"	t_{hbp}	216	NCLK
Horizontal "Front Porch"	t_{hfp}	40	NCLK
Tiempo Habilitación DEN	t_{ep}	800	NCLK

La Tabla muestra los distintos valores temporales de la temporización horizontal para una resolución de imágenes de 800 por 480 pixeles. El fabricante establece que la frecuencia de reloj optima para su funcionamiento debe ser de 33,2 MHz. Sin embargo, se pueden utilizar otras frecuencias de reloj, estableciendo un máximo en 40 MHz. No obstante, se puede conseguir una visualización sea correcta de las imágenes, utilizando frecuencias de reloj de 25 MHz.

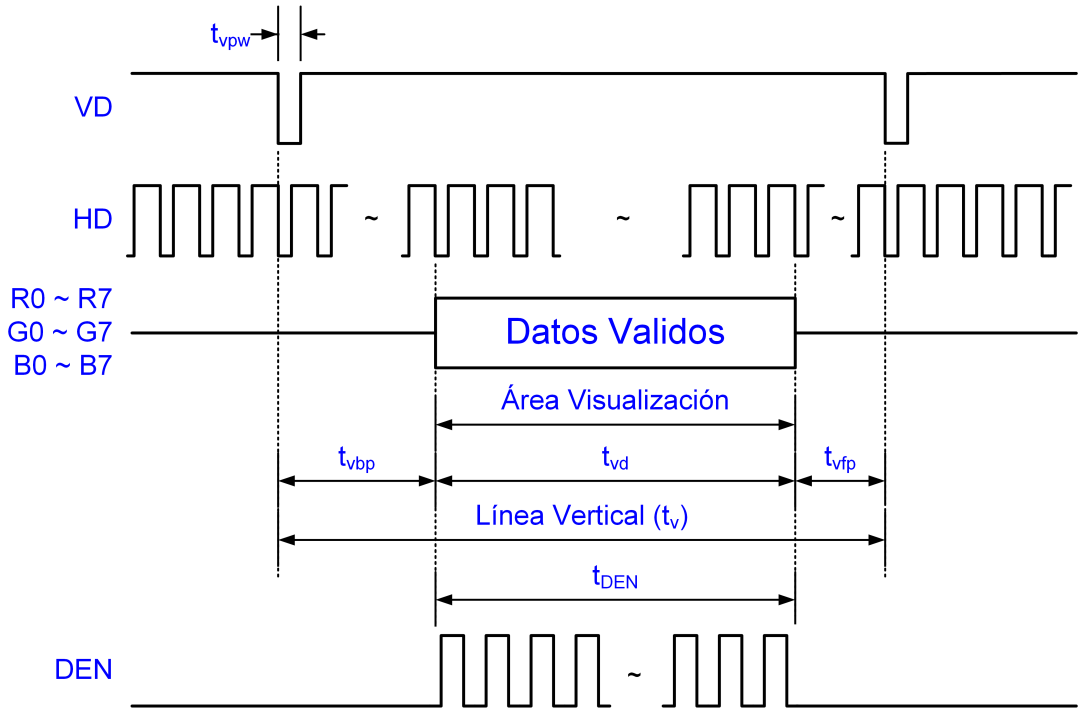
The Table shows the different time values used for horizontal timing with an image resolution of 800 × 480 pixels. According to the manufacturer, the optimal clock frequency is 33,2 MHz, however it is possible to use other clock frequencies up to a maximum of 40 MHz. In particular, it is possible to achieve correct image display using a 25 MHz clock frequency.

Diagrama Temporización Vertical

Vertical Timing Diagram

En la Figura se muestran los requerimientos temporales básicos para la visualización de cada "*frame*" o imagen (vertical) que se quiere mostrar en la pantalla. La temporización del sincronismo vertical (VD), es similar a la horizontal a excepción que un pulso de sincronismo vertical significa que ha terminado un "*frame*" o refresco completo de la pantalla y comienza el siguiente. Además, los datos se refieren al conjunto de todas las filas de un "*frame*".

The Figure below shows the basic timing specifications for the display of each "frame" or (vertical) image to be represented on screen. The vertical sync signal (VD) is similar to the horizontal sync, but pulses are now used to indicate the end of a frame (i.e. complete screen refresh) and the start of a new one. Moreover the "valid data" now refer to the collection of all rows in a frame.



Un pulso activo a nivel bajo de duración concreta (t_{vpw}), se utiliza como señal de sincronismo vertical (VD), para indicar el final de una imagen o "*frame*" y el comienzo de la siguiente. Después del pulso de sincronismo vertical, las señales de datos (RGB), todavía no son válidas durante un periodo de tiempo conocido como vertical "*back porch*" (t_{vbp}). Tras este tiempo, vendrá el tiempo de visualización de los datos en pantalla (t_{vd}), en el que los datos se aplican a los pixeles, pero únicamente cuando se este dentro de la zona de visualización indicada por las señales horizontales. Durante el tiempo de visualización de los datos en la pantalla (t_{vd}), la señal de habilitación de la pantalla (DEN) va cambiando estando desactivada (a nivel bajo) durante el horizontal "*back porch*" y el "*front porch*". Finalmente hay un nuevo periodo temporal denominado vertical "front porch" (t_{vfp}), en el que las señales de datos (RGB) vuelven a no ser válidos, y que tras concluir se produce un nuevo pulso de sincronismo vertical.

Es importante destacar, que la señal de sincronismo horizontal (HD), actúa habilitando los cambios de las señales mostradas en este diagrama, ya que los cambios se producen en esta ocasión al cambiar de línea por ser una variación vertical del puntero de visualización.

An active low pulse of fixed length (t_{vpw}) is used as the vertical sync signal (**VD**) in order to determine the end of an image or *frame* and the start of the next one. After the vertical sync pulse, the data signals (RGB) are invalid for a period of time known as the vertical "back porch" (t_{vbp}). After this we have the time for onscreen data display (t_{vd}), when the data are applied to the pixels only within the display area determined by the horizontal signals. During this data display time (t_{vd}), the Display Enable signal (**DEN**) remains inactive (low) during the horizontal "back porch" and "front porch". Finally, there is another time interval called vertical "front porch" (t_{vfp}) when the RGB data signals are again invalid. After this time interval, the new vertical sync pulse is generated.

It is important to notice that all changes in the signals in this diagram are timed by the horizontal sync signal (**HD**), since they only take place when changing rows as they correspond to vertical changes in the display pointer.

Parámetro	Símbolo	Valor	Unidad
Datos Verticales	t_{vd}	480	H
Total Periodo Vertical	t_{v}	525	H
Ancho de Pulso VS	t_{vpw}	1	H
Vertical “Back Porch”	t_{vbp}	35	H
Vertical “Front Porch”	t_{vfp}	10	H
Vertical “blanking”	t_{vb}	45	H
Tiempo Habilitación DEN	t_{DEN}	480	H

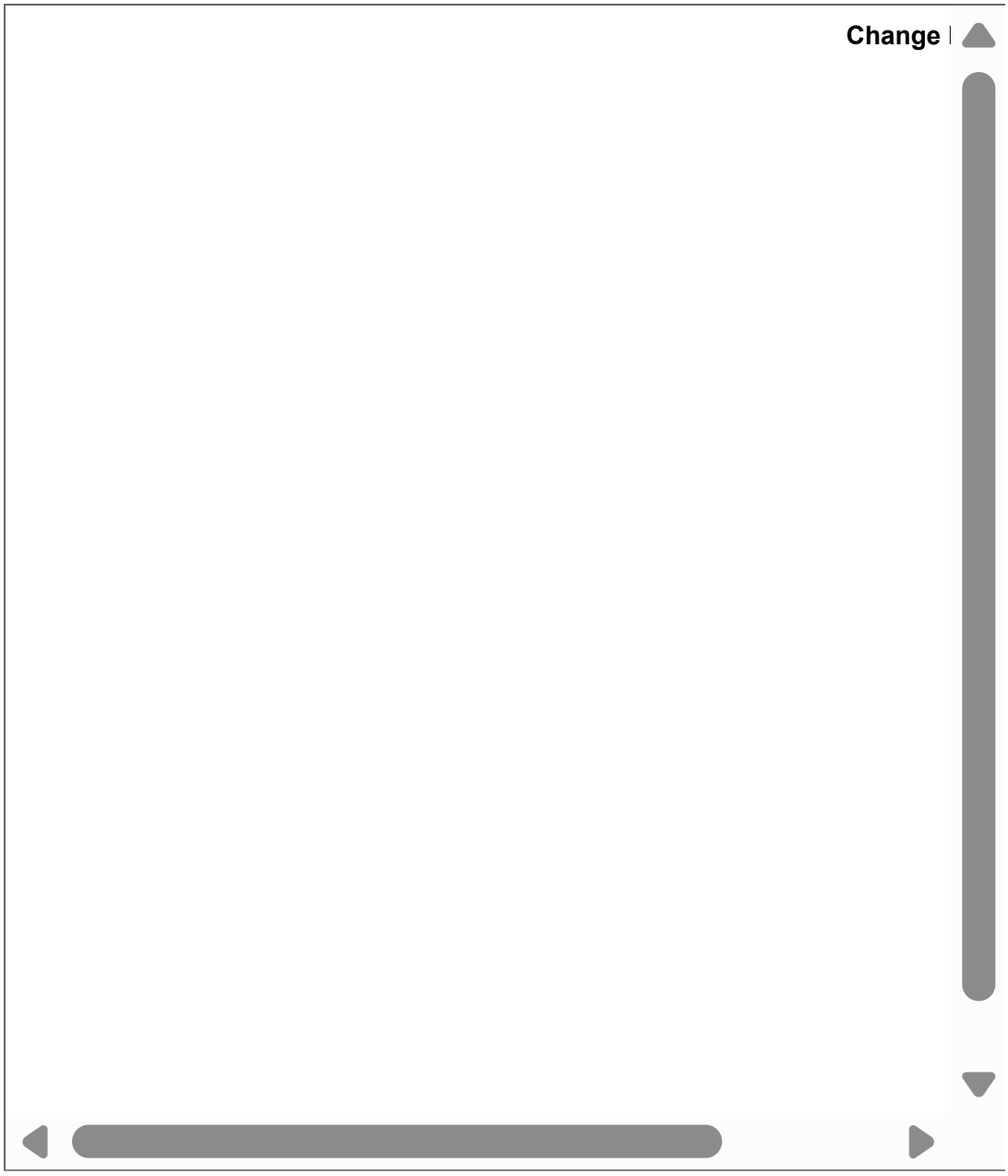
CLa Tabla muestra los distintos valores temporales de la temporización vertical para una resolución de imágenes de 800 por 480 pixeles. Los valores se establecen tomando como unidad el tiempo que se tarda en recorrer una línea horizontal. Por ejemplo, el dato del vertical “*back porch*” tiene un valor de 35 líneas horizontales.

The Table shows the different time values used for vertical timing with an image resolution of 800 × 480 pixels. Values are given in terms in terms of a time unit (**H**) that represents the time taken to scan a horizontal line. For instance, the vertical “back porch” is equal to 35 horizontal lines.

Estructura de la Tarea

Project Structure

ABRIR EN UNA VENTANA NUEVA



La práctica se centra en la realización de un diseño que implemente un sistema de control y visualización en una pantalla. En este caso particular se ha escogido una pantalla táctil para su aplicación, aunque la pantalla permite obtener las coordenadas del lugar donde se ha tocado, en está práctica solo se centrará en la parte de la gestión de la visualización.

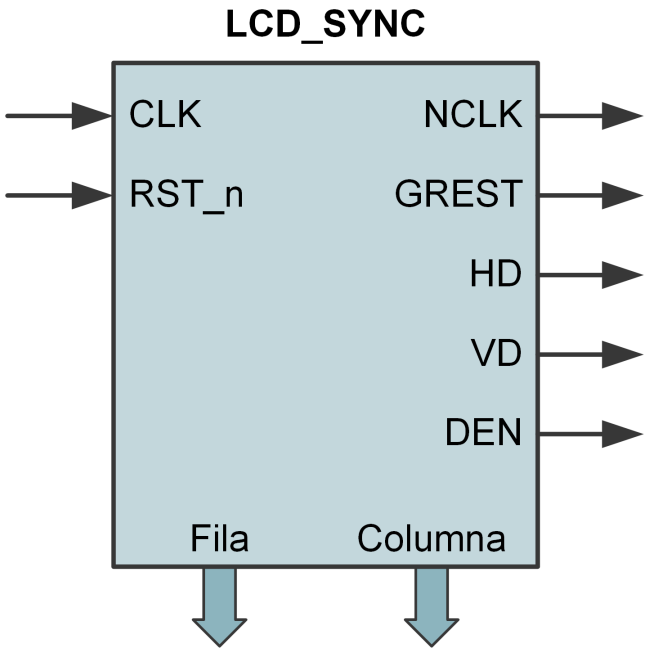
La práctica se divide en siete sub-tareas, la primera de ellas consiste en realizar el diseño del generador de sincronismo para poder visualizar en la pantalla, a partir de aquí las sub-tareas se van incrementando en complejidad, añadiendo colores a la visualización, barras, caracteres, hasta alcanzar la escritura de caracteres en la pantalla proveniente de un teclado PS2 conectado al hardware.

This project focuses on the design of a system for control of and display on a screen. We have chosen a touchscreen as a particular application case, which allows us to obtain the coordinates where the screen is touched. But this project will only consider the display aspects of the screen.

The project is divided into seven sub-tasks. The first one consists in designing the synchronism generator for the rest of display signals. From that point on, subsequent sub-tasks increase in complexity and incorporate the display of colors, bars, characters, until finally we are able to write on screen the characters coming from a PS2 keyboard connected to our hardware.

Subtarea 1: Generación de las señales de sincronismo

Sub-task 1: Generating synchronism signals


LCD_SYNC
CLK
RST_n
NCLK
GREST
HD
VD
DEN
Fila
Columna

En primer lugar se aborda el diseño del bloque encargado de generar las señales de datos y sincronismo de la pantalla. Este bloque permitirá la representación de imágenes en la pantalla. Es importante destacar que la configuración inicial de la pantalla permite la visualización de imágenes con una resolución de 800 × 480 pixeles; lo cual significa que no es necesario realizar el diseño del interface serie de la pantalla, para poder ver imágenes en la pantalla. El interface serie de la pantalla sólo hace falta en el caso de querer cambiar la resolución de la pantalla u otros parámetros de configuración o de la visualización de la misma.

Se quiere diseñar el módulo que genere las señales de sincronismo de la pantalla. Tal como se observa en la Figura, este módulo tendrá las siguientes señales: como entradas una señal de reloj CLK y una señal de reset activa a nivel bajo RST_n y como salidas se deben generar las señales de sincronismo horizontal (HD) y vertical (VD), la señal de reset global de la pantalla (GREST), la señal de reloj de salida hacia la pantalla (NCLK), la señal de habilitación de la visualización en la pantalla (DEN) que sirve para indicar cuando las señales de datos RGB son validas y finalmente dos señales (en este caso se trata de buses o arrays) denominadas Fila y Columna que servirán para indicar en cada momento el valor de la posición o coordenadas del pixel que se va representar en la pantalla.

Let us first tackle the design of the block that generates the data and synchronism signals for our screen. This block will enable the display of images on the screen. We point out here that the default initial configuration of the screen allows the display of images with a resolution of 800 × 480 pixels, so it is not necessary to implement the serial interface for screen configuration in order to be able to display images. This interface is only needed if we want to change the screen resolution or other configuration or display parameters.

The task consists in designing a module that generates the screen's synchronism signals. As can be seen on the Figure, this module will have the following ports:

Inputs: A clock signal (**CLK**) and a reset signal, active low and asynchronous (**RST_n**).

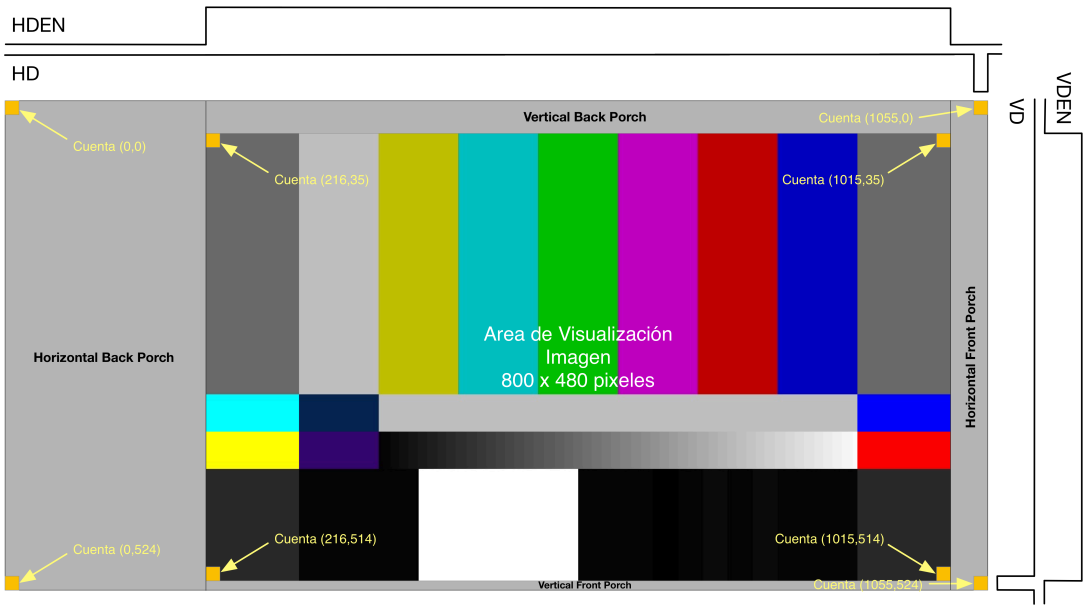
Outputs: The horizontal sync (**HD**) and vertical sync (**VD**) signals, the screen global reset signal (**GREST**), the clock output to the screen (**NCLK**), and the display enable signal (**DEN**) that indicates when the RGB data signals are valid. Finally, two signals -buses in this case- called Row (**Fila**) and Column (**Columna**) that contain the position/coordinates of the pixel to be represented on screen at any given time.

Representación con Cuentas

El diseño del controlador de la pantalla, básicamente consiste en generar las señales de sincronismo horizontal y vertical, así como la señal de reloj necesaria que permitan guiar el barrido del punto o "*dot clock*" por la pantalla de manera adecuada. Para ello hay que tener presente los diagramas temporales con sus parámetros que nos fija el fabricante de la pantalla. A partir de ellos se puede determinar que para visualizar una imagen de 800 × 480 pixeles, se tiene que realizar o gestionar el barrido a través de 1056 × 525 puntos. En la Figura se representan las coordenadas de los puntos extremos que hay que recorrer y que posiciones ocupará el área de visualización.

Representation using Counters

The design of the screen controller consists basically in generating the horizontal and vertical sync signals, as well as the clock signal required by the screen to control the dot clock's scan speed across the screen. In order to do that, we must keep in mind the timing diagrams given by the screen's manufacturer and their parameters. Based on that information we can conclude that, in order to display an image of 800 × 480 pixels, we must actually implement a scan across a grid of 1056 × 525 positions. The Figure below shows the coordinates of the extreme points in this grid and the positions corresponding to the valid display area.



El área de visualización comenzará en la coordenada (216, 35) y tendrá el otro extremo de la diagonal en la coordenada (1015, 514); entre ellas se ubican los 800 por 480 pixeles. En estos rangos de coordenadas se puede observar que se encuentran activas las dos señales HDEN y VDEN, para indicar que se trata el área de visualización de la imagen.

The display area begins at the coordinates (216,35) and ends at the coordinates (1015,514). The 800 × 480 display pixels are located between them. The signals HDEN and VDEN are active in these coordinate ranges in order to indicate that they correspond to the image display area.

Propuesta Estructura Diseño

Proposed Design Structure

Se puede plantear el diagrama de bloques de la imagen como propuesta orientativa para realizar el diseño.

Comenzando por la parte superior izquierda, se encuentra un administrador de reloj o también conocido como PLL, que generará la señal de reloj de 25 MHz necesaria para la pantalla NCLK, a partir del oscilador externo de 50 MHz. El resto del diseño debe funcionar con esta señal de reloj de 25 MHz generada por el PLL.

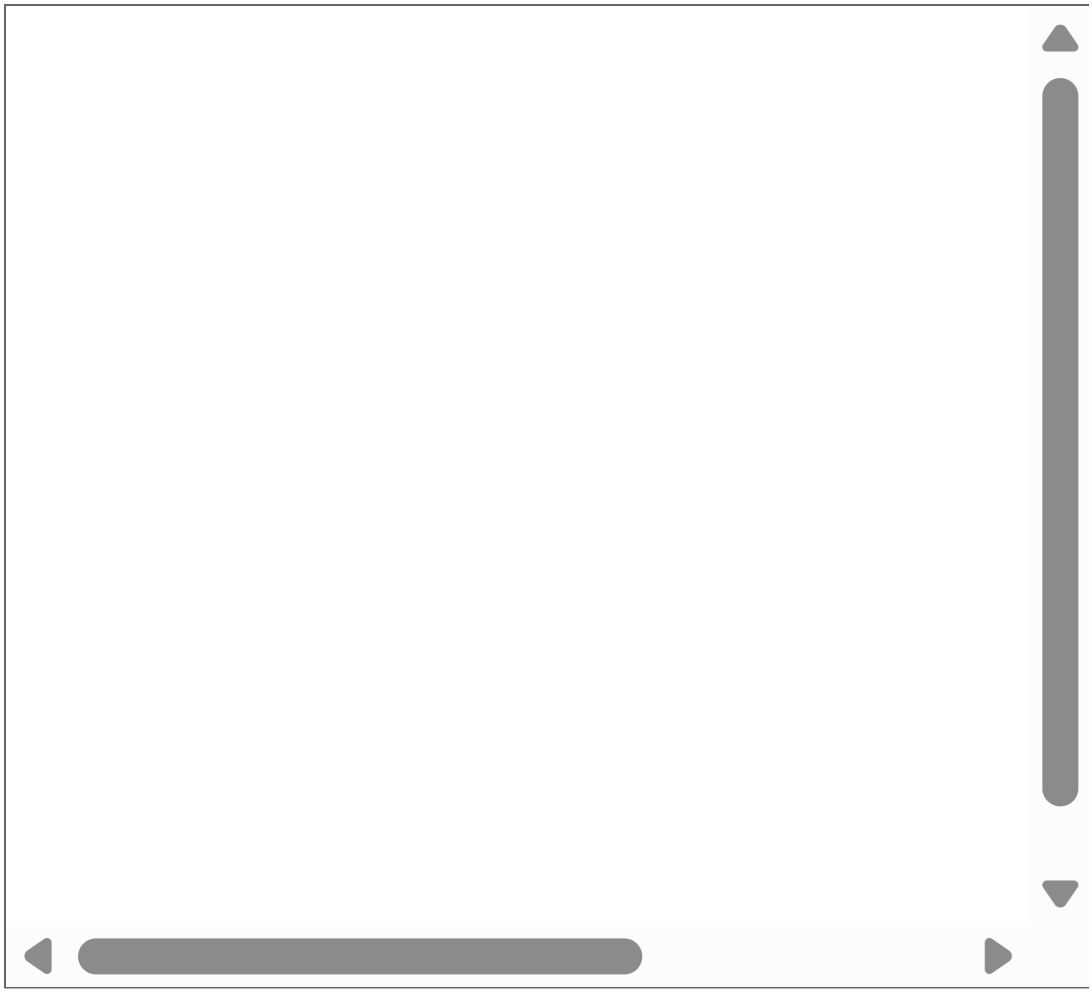
A continuación en la parte central, destaca el contador de modulo 1056, que permite contar el número de pixeles horizontales, o el número de columnas de la pantalla. En cada flanco de subida de la señal de reloj de 25 MHz se incrementa la cuenta, indicando así el valor de la coordenada horizontal. Cuando el contador llegue a la cuenta 1055 deberá volver a cero. El contador dispone de una salida de fin de cuenta TC que se debe activar cuando la cuenta vale 1055, además se utilizará para generar la señal de sincronismo horizontal.

As a suggestion for this sub-task, we propose the block diagram in the image below.

Starting from the top left corner, there is a clock management subsystem (PLL) that will generate the 25 MHz clock signal for the screen (NCLK) from the external 50 MHz oscillator. The rest of the design will operate on this 25 MHz clock signal from the PLL.

At the center of the image there is a counter modulo 1056 (HCOUNT) that will allow us to count the horizontal pixel number (or, equivalently, the column number). The count, and thus the horizontal coordinate value, is incremented on each rising edge of the 25 MHz clock. After the counter reaches the value 1055 it must go back to 0. The counter also features a terminal count (TC) signal that activates when the count reaches 1055 and will also be used to generate the horizontal sync signal.

ABRIR EN UNA VENTANA NUEVA



En la parte inferior central se encuentra el contador de modulo 525, que permite contar el número de pixeles verticales, o el número de filas de la pantalla. Cada vez que el contador HCOUNT llegue al final de la cuenta, hará que se incremente la cuenta de este contador, indicando así el valor de la coordenada vertical. La señal de fin de cuenta TC, se utilizará para generar la señal de sincronismo vertical.

Justo en el lado derecho y de color amarillo se encuentra el generador de la señal DEN, que partiendo de los valores de la columna y de la fila suministrados por los contadores HCOUNT y VCOUNT, generará la señal DEN. Esta señal debe estar activa durante el área de visualización de la pantalla, lo cual corresponde a las coordenadas comprendidas desde la (**216**, **35**) a la (**1015**, **514**), tal como se ha visto anteriormente.

Por último el uso de dos inversores, permitirán obtener las señales de sincronismo horizontal HD y de sincronismo vertical VD, a partir de las señales de fin de cuenta de los contadores HCOUNT y VCOUNT, respectivamente.

Immediately below we also find a counter modulo 525 (VCOUNT) that allows us to count the vertical pixel number (or, equivalently, the row number). Whenever the HCOUNT counter reaches its terminal count, it causes the VCOUNT counter to increment its count and thus the vertical coordinate. The terminal count (TC) signal will be used to generate the vertical sync signal.

At the right margin, and marked in yellow, we have the DEN signal generator. This signal will be obtained based on the column and row values provided by the HCOUNT and VCOUNT counters. The DEN signal must be active on the screen display area, which corresponds to the coordinates between **(216, 35)** and **(1015, 514)** as explained before.

Finally, two inverters will allow us to obtain the horizontal (HD) and vertical (VD) sync signals from the terminal count signals of HCOUNT and VCOUNT, respectively.

Usando el IP Catalog para Crear un PLL

Using IP Catalog to Instantiate a PLL

Es relevante destacar que aunque la señal de reloj NCLK, la establezca el fabricante de la pantalla en 33,2 MHz, las especificaciones de la pantalla acepta frecuencias hasta 40 MHz, siendo perfectamente válida la opción de utilizar una señal de 25 MHz, al igual que la de una VGA. En este caso lo más adecuado es que todo el diseño funcione con una señal de reloj de 25 MHz, en lugar de los 50 MHz del oscilador de la placa DE2-115.

¿Cómo se pueden obtener los 25 MHz, teniendo en cuenta que el oscilador externo es de 50 MHz y mantener los criterios del correcto diseño síncrono? La respuesta es: mediante un PLL de los que disponen internamente las FPGAs. Este elemento permite generar un reloj, de distinta frecuencia y fase, a partir de otro. Los PLLs internos del dispositivo se pueden utilizar generándolos y caracterizándolos mediante la herramienta **IP Catalog (antes MegaWizard Plug-In Manager)** de Quartus II. El **IP Catalog** es una biblioteca de bloques hardware y funciones sintetizabas, que se pueden utilizar en los diseños de Quartus. Entre ellas se dispone de una función denominada **ALTPLL** que permite crea un nuevo módulo con los parámetros adecuados para utilizar las PLL internas de la FPGA. Por medio de las diferentes ventanas de configuración de la función **ALTPLL** se establecen las características del PLL que se quiere utilizar. Dicha función genera los archivos necesarios para utilizar el PLL, entre ellos una plantilla para la instanciación del mismo en el diseño donde se vaya a utilizar.

Although the screen's manufacturer sets the nominal frequency for the NCLK clock signal at 33.2 MHz, the screen specifications admit frequencies up to 40 MHz, so it is perfectly valid to use a 25 MHz signal as in a VGA screen. In our case, the most appropriate choice is to have the whole design operate on a 25 MHz clock signal instead of the 50 MHz offered by the DE2-115's on board oscillator.

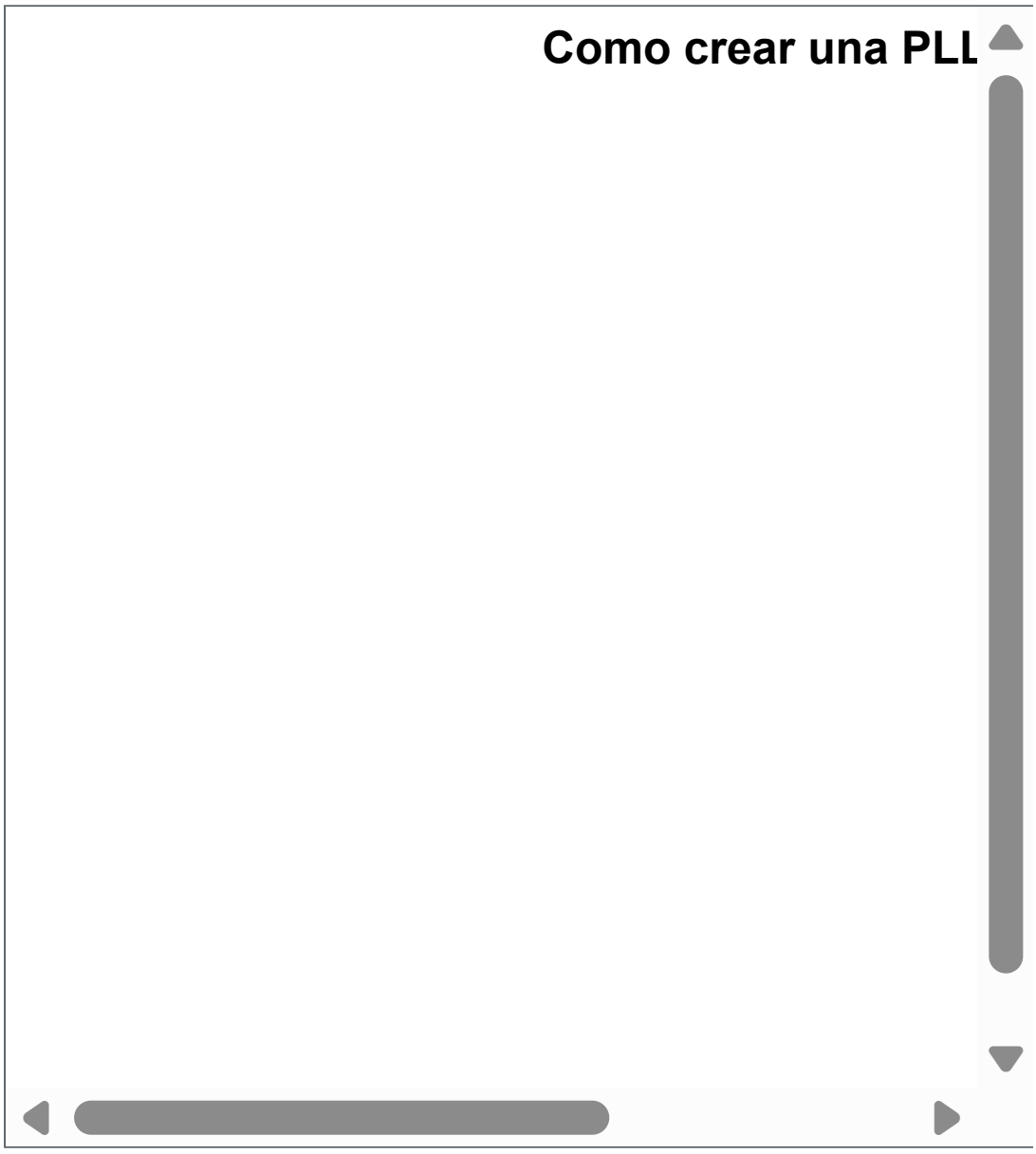
How can we obtain a 25 MHz clock if our external oscillator provides 50 MHz, and at the same time comply with the criteria for correct synchronous design? The answer is: using one of the PLLs typically found inside of FPGAs. This element allows us to generate a clock signal from another clock signal, with different frequency and/or phase. It is possible to use the FPGA's internal PLLs by generating and characterizing them with the **IP Catalog** (previously, "*MegaWizard Plug-In Manager*") tool in Quartus. **IP Catalog** is a library of hardware blocks and synthesizable functions that may be used for Quartus designs. Among them, there is a block called **ALTPLL** that allows us to create a new module with the appropriate parameters in order to use the FPGA's internal PLLs. The different configuration windows of the **ALTPLL** block allow us to select the parameters and features of the PLL to be used. The tool then generates the necessary files to use the PLL, among them a template for instantiation of the PLL in our design.

The Figure below, titled **How to create a PLL**, contains an animated guide to the generation of the PLL, the para-

En la Figura inferior denominada **Como crear una PLL**, se puede ver una guía animada de cómo generar el PLL, que parámetros hay que indicar y que opciones y ajustes establecer para a partir de un reloj de 50 MHz de entrada genere un reloj de 25 MHz. El punto de partida para la utilización del **IP Catalog**, es el proyecto de Quartus donde se realice el diseño.

meters and options to be selected and the adjustments to be made in order to generate a 25 MHz clock output from a 50 MHz clock input. The starting point for **IP Catalog** use is our Quartus project where the design is being implemented.

ABRIR EN UNA VENTANA NUEVA



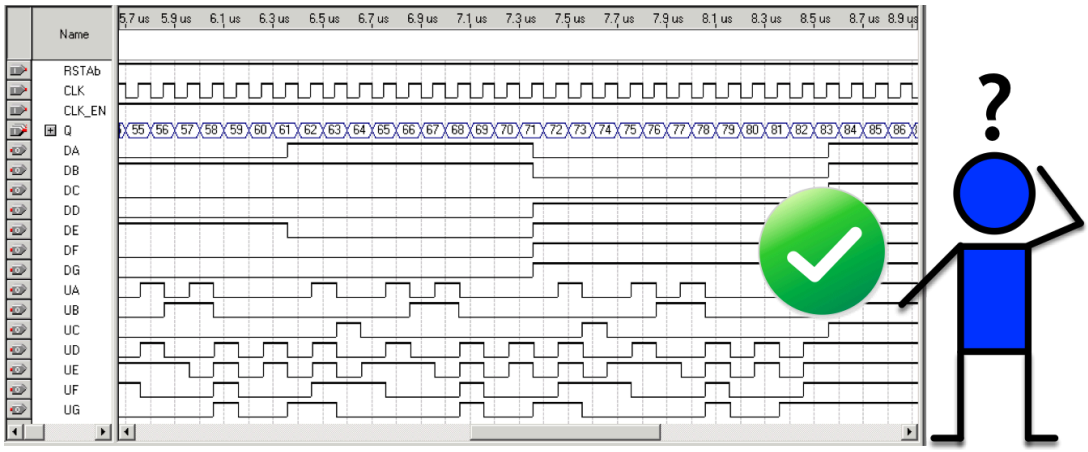
Como resultado de la generación del PLL, el **IP Catalog**, habrá creado en el directorio del proyecto una serie de archivos, entre los cuales se encuentra el denominado **pll_ltm_inst.v**. Este archivo contiene una descripción ejemplo de cómo realizar la instanciación del PLL generado en el diseño para poderlo utilizar. Su contenido se muestra en la siguiente imagen:

As a result of PLL generation, **IP Catalog** will create a series of files in our project folder, and in particular one called **pll_ltm_inst.v**. This file contains an example description showing how to instantiate the generated PLL in our design. Its contents are shown below:

```
pll_ltm      pll_ltm_inst (
  .inclk0 ( inclk0_sig ),
  .c0 ( c0_sig )
);
```

Verificación del Diseño

Design Verification



Una vez realizado el diseño en Verilog HDL del módulo LCD_SYNC, hay que verificar su funcionamiento mediante las correspondientes simulaciones. Es necesario comprobar que los contadores funcionan correctamente y se generan adecuadamente y según las necesidades las señales de sincronismo horizontal y vertical. Una comprobación mínima, requiere recorrer la pantalla al menos una vez. Esto significa un tiempo de simulación cercano a los 23 ms.

Realizar un Testbench en Verilog HDL para la simulación que mediante el uso de "Task", utilice el valor de Fila y Columna y/o de las señales de sincronismo para detener la simulación. En primera instancia la simulación puede detenerse tras recorrer la primera Fila de la pantalla, tras lo cual se puede realizar una simulación integra de todo el recorrido de la pantalla. En esta ocasión no se puede utilizar el tiempo para detener la simulación.

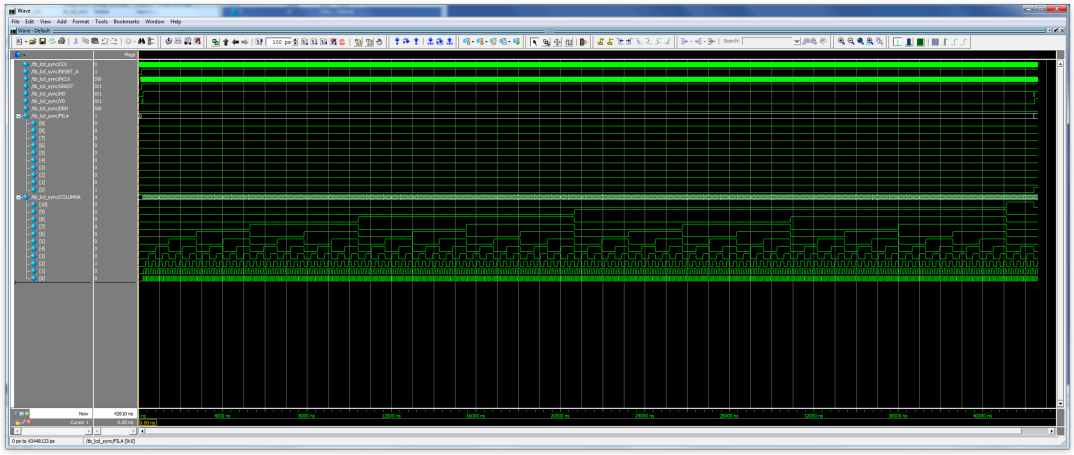
Tras la simulación se puede comprobar el buen funcionamiento del diseño en el visualizador de ondas. Utilizando las opciones de zoom representado en el menú mediante las lupas, se puede manejar la visualización para comprobar que los contadores y las señales de sincronismo horizontal y vertical funcionan correctamente.

Once the Verilog HDL design of the LCD_SYNC module has been finished, we must verify its correct operation using the corresponding simulation. It is necessary to check that the counters work correctly and the horizontal and vertical sync signals are generated adequately according to specifications. At the very least, the simulation should cover one complete scan of the whole screen. This corresponds to a simulation time of 23 ms approximately.

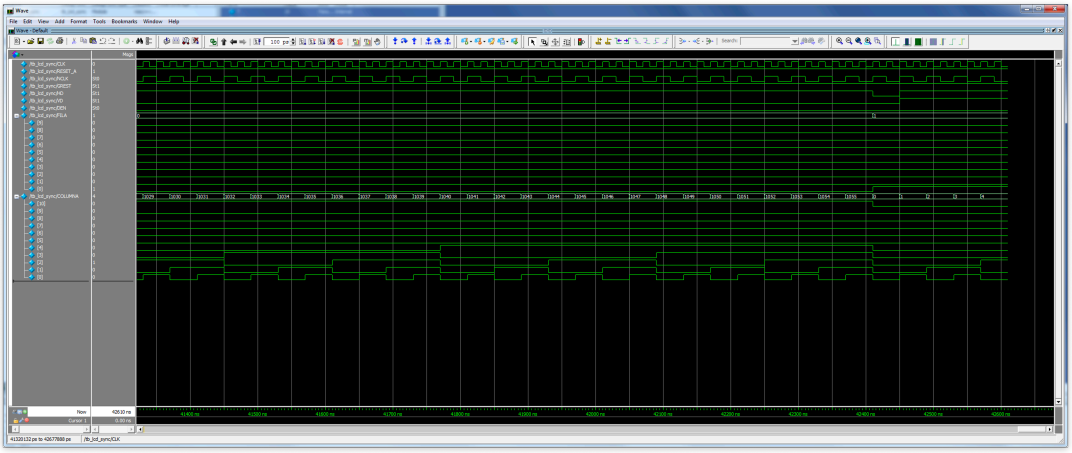
We must write a Testbench for simulation using Verilog HDL, based on the use of tasks, such that it is capable of stopping the simulation based on the Row and Column values and/or the sync signals. One suggestion is to first try to make the simulation stop after scanning the first row, and then make the simulation stop after scanning the whole screen. We must NOT use fixed time intervals to stop the simulation.

After the simulation, we should be able to verify the correct operation in the waveform viewer. Use the zoom options (represented by the magnifying glass icons) to manage the waveform display in order to check that the counter values and the horizontal and vertical sync signals are correct.

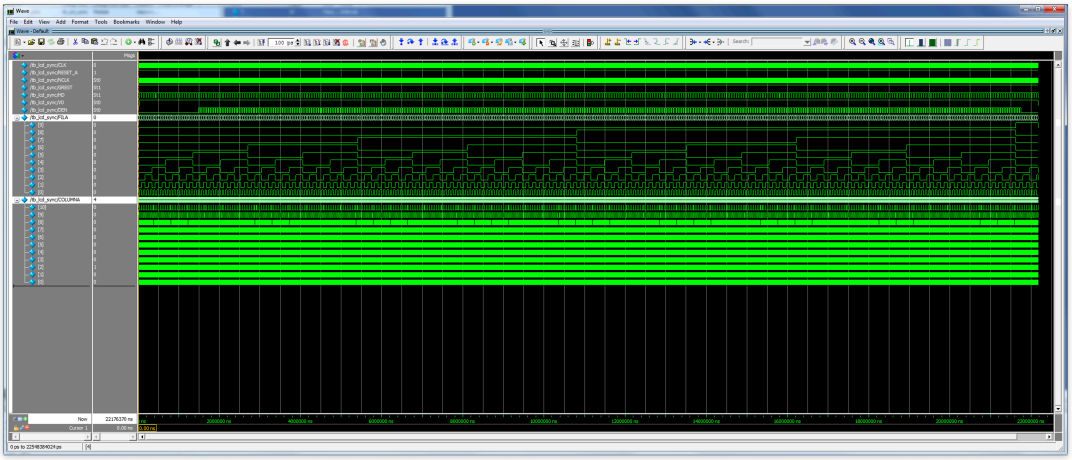
Simulación de una Fila / Simulation for one row:



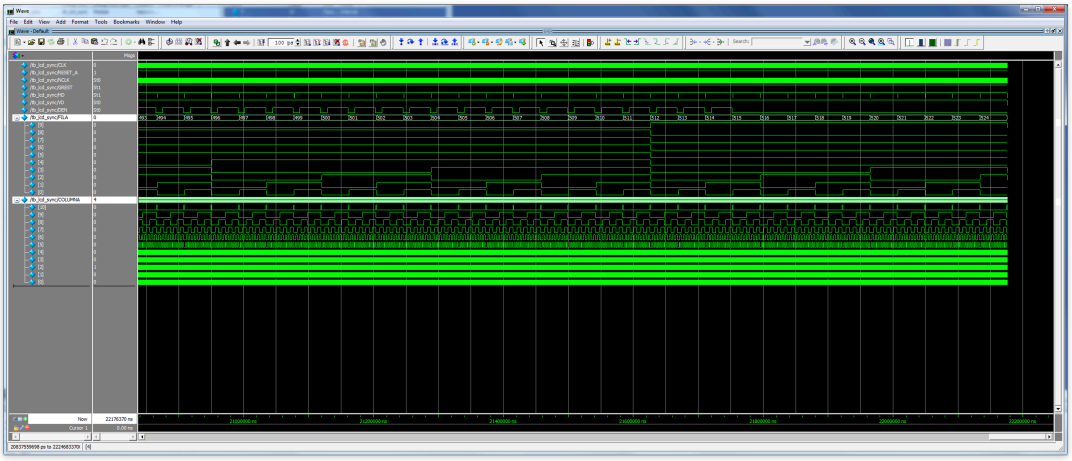
Detalle final simulación una Fila / End of simulation for one row (detail):



Simulación un recorrido por la pantalla completo / Simulation of a complete screen scan:

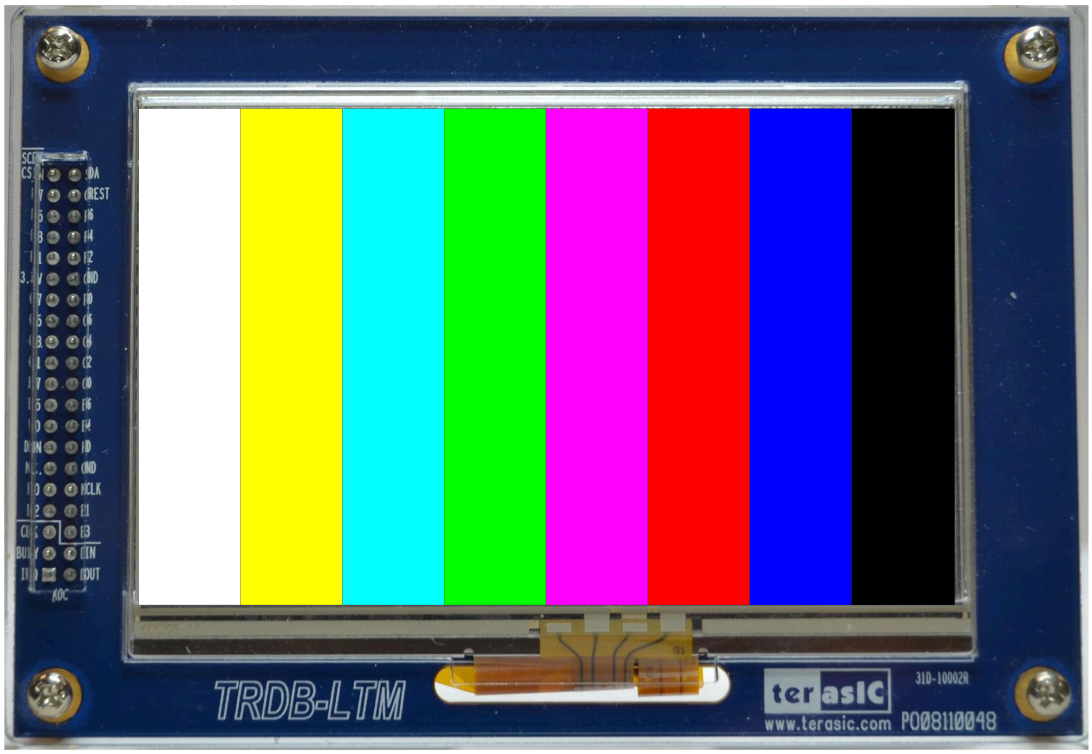


Detalle final simulación un recorrido por la pantalla completo / End of simulation of a complete screen scan (detail):



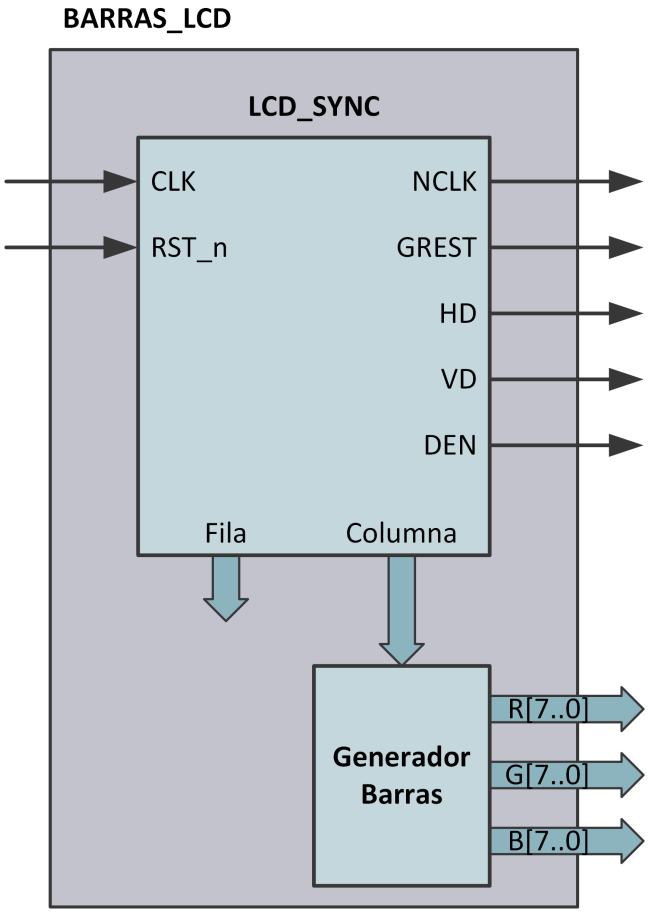
Subtarea 2: Generación de barras de colores en la pantalla

Sub-task 2: Displaying color bars on the screen



Tras diseñar el módulo de generación de las líneas de control LCD_SYNC, ahora se requiere verificar su funcionamiento en la placa DE2-115. En este apartado únicamente se requiere utilizar el módulo LCD_SYNC y asignar un valor a las señales correspondientes de los colores (RGB), para que en la pantalla se vean las barras de colores de la imagen superior. En la Figura inferior se muestra el esquema de bloques de lo que hay que realizar. Notar que las barras de colores tienen todas la misma anchura y por tanto el valor de las señales de color RGB, dependerán únicamente del valor de la columna generado por el bloque LCD_SYNC.

Once the module LCD_SYNC for generation of control signals has been designed, we have to test its operation on the DE2-115 board. In order to do that, in this sub-task we only need to use the LCD_SYNC block and assign a value to the color data signals (RGB) so that the color bars from the image above are displayed on the screen. The Figure below contains a block diagram of the sub-task. Note that all color bars have the same width and the value of the RGB color signals will only depend on the column value given by the LCD_SYNC block.



Una vez realizado el diseño del módulo BARRAS_LCD, hay que verificar su correcto funcionamiento mediante una simulación RTL en ModelSim. Sin embargo, intentar comprobar si el funcionamiento es correcto mediante el uso del visor de ondas de ModelSim puede ser un trabajo laborioso. Afortunadamente en esta ocasión se dispone de una herramienta web que emula el comportamiento de la pantalla, mediante la generación de una imagen, a partir de un fichero de texto y de la configuración adecuada de los parámetros de la pantalla. El fichero de entrada al emulador es el resultado de una simulación realizada con ModelSim en el que se indica en cada línea: el tiempo de simulación y los valores en ese momento de las señales HD, VD, DEN, R, G y B.

A continuación, se muestra un ejemplo de las primeras líneas del archivo de texto, para una codificación de RGB de 24 bits que emplea 8 bits para color. Ver que el tiempo de simulación se indica en picosegundos, pero también se puede utilizar en nanosegundos.

After the BARRAS_LCD module design is finished, we should verify its correctness with a RTL simulation on ModelSim. However, trying to check whether it works properly using the ModelSim waveform viewer can be a rather long task. Fortunately, a web tool is available that emulates the behavior of the screen by generating an image from a text file and a specification of the screen parameters. The file used as input to the emulator will be generated using a ModelSim simulation. In each line, it contains the simulation time and the current values of the following signals: HD, VD, DEN, R, G, B.

Let us see an example of the first lines of this text file for a 24-bit RGB encoding that uses 8 bits per color. Note that the simulation time is specified in picoseconds, but it is also possible to do it in nanoseconds.

Codificación RGB 24 bits

```
10000 ps: 0 1 0 11111111 11111111 11111111
30000 ps: 0 1 0 11111111 11111111 11111111
50000 ps: 0 1 0 11111111 11111111 11111111
70000 ps: 0 1 0 11111111 11111111 11111111
90000 ps: 0 1 0 11111111 11111111 11111111
110000 ps: 0 1 0 11111111 11111111 11111111
130000 ps: 0 1 0 11111111 11111111 11111111
150000 ps: 0 1 0 11111111 11111111 11111111
170000 ps: 0 1 0 11111111 11111111 11111111
190000 ps: 0 1 0 11111111 11111111 11111111
210000 ps: 0 1 0 11111111 11111111 11111111
…
```

Para obtener un fichero de texto con las señales indicadas anteriormente, hay que hacer una simulación del diseño en ModelSim. Por supuesto, es necesario escribir un testbench para hacer la simulación, tal como se ha visto en las clases anteriores. En este caso hay que añadir al testbench, los bloques procedurales indicados a continuación, con los pertinentes ajustes al diseño de cada uno. Con esto se obtiene el archivo de texto `vga.txt`, que contiene los resultados de la simulación en el formato adecuado. La simulación, debe ser de suficiente tiempo para ver un frame completo, lo cual equivale a casi 23 ms.

In order to obtain a text file as shown above, we have to simulate the design on ModelSim. Of course, it will be necessary to write a testbench for that simulation, as has been already done in previous lab sessions. In this occasion, the following procedural blocks have to be added to the testbench (with the appropriate adjustments to your particular design). This will generate a text file called `vga.txt` containing the simulation results with the appropriate format. The simulation must be long enough to display a complete frame, i.e. at least 23 ms approximately.

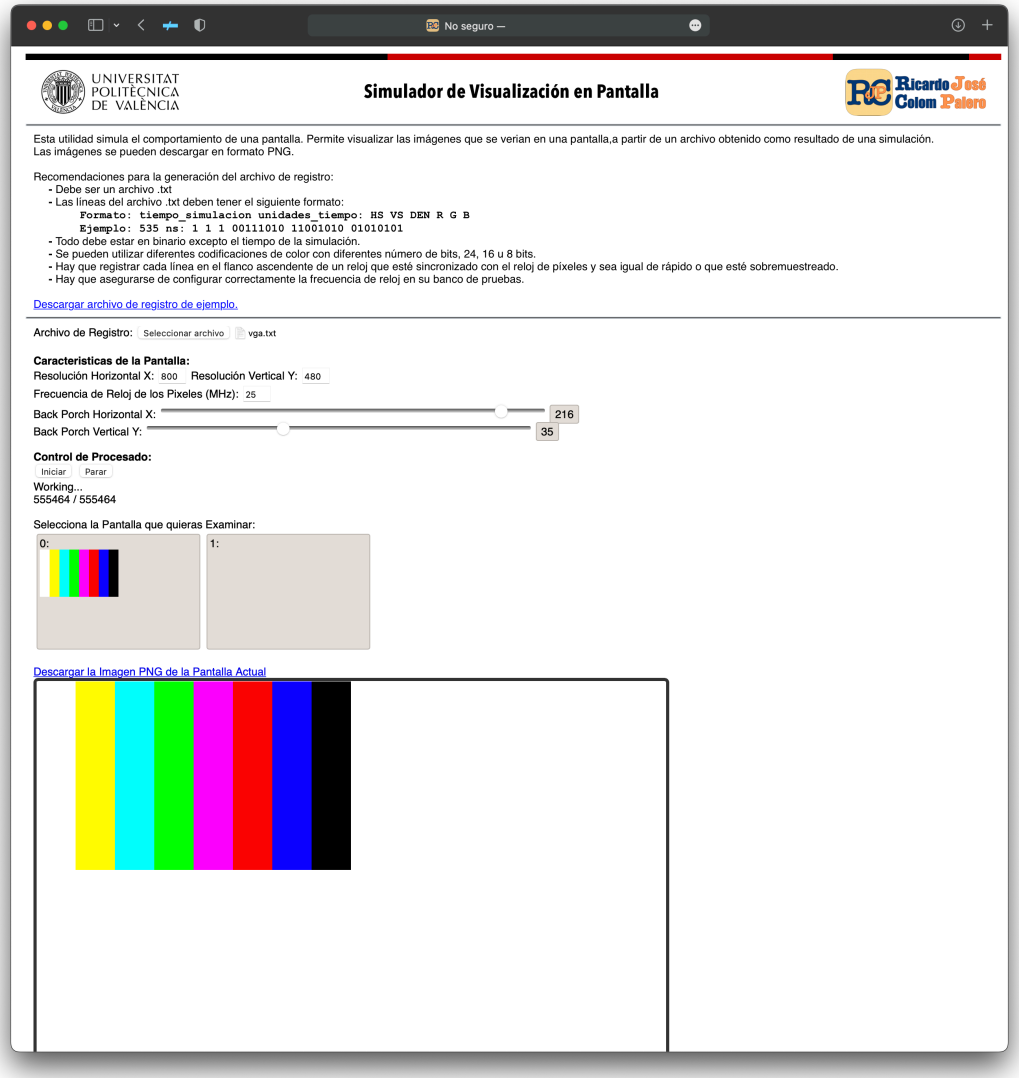
```
integer fd;
event cierraFichero;

initial
// Vuestra simulación. Todos los casos de test
…
   @(posedge VD);
   $display("Fin de la simulacion\n");
   -> cierraFichero;
   #10;
   $stop;
end

initial begin
   fd = $fopen("vga.txt","w");
   @(cierraFichero);
   disable guardaFichero;
   $display("Cierro Fichero");
   $fclose(fd);
end

initial forever begin: guardaFichero
   @(posedge CLK50) //o NCLK, para obtener un archivo más pequeño
   $fwrite(fd,"%0t ps: %b %b %b %b %b %b\n",$time,HD,VD,DEN,R,G,B);
end
```

Para verificar que el resultado de la simulación con ModelSim es correcto, hay que utilizar el emulador de pantalla (Screen Simulator), que se encuentra en la barra lateral del PoliformaT o directamente en la dirección: SCREEN SIMULATOR.

Tras seleccionar el archivo resultado de la simulación `vga.txt`, pulsar el botón `Iniciar` , se pondrá a procesar el archivo y se obtendrá el resultado de la visualización tal como se muestra en la siguiente imagen.

In order to verify that the ModelSim simulation result is correct, use the Screen Simulator. It can be accessed through the left menu bar in Poliformat, or directly at the address SCREEN SIMULATOR.

After selecting the simulation output file `vga.txt` and clicking on the `Iniciar` button, the file will be processed and the display result will be obtained as in the following image.



Una vez comprobado que el resultado de la simulación es correcto, se puede pasar a probar el diseño en la placa DE2-115. Para ello, se tendrá que realizar la asignación de pines correspondiente tal como se muestra en la siguiente tabla.

Once the correctness of the simulation result is assessed, the design can be tested on the DE2-115 board. In order to do that, the pin assignments in the following table must be added to the design:

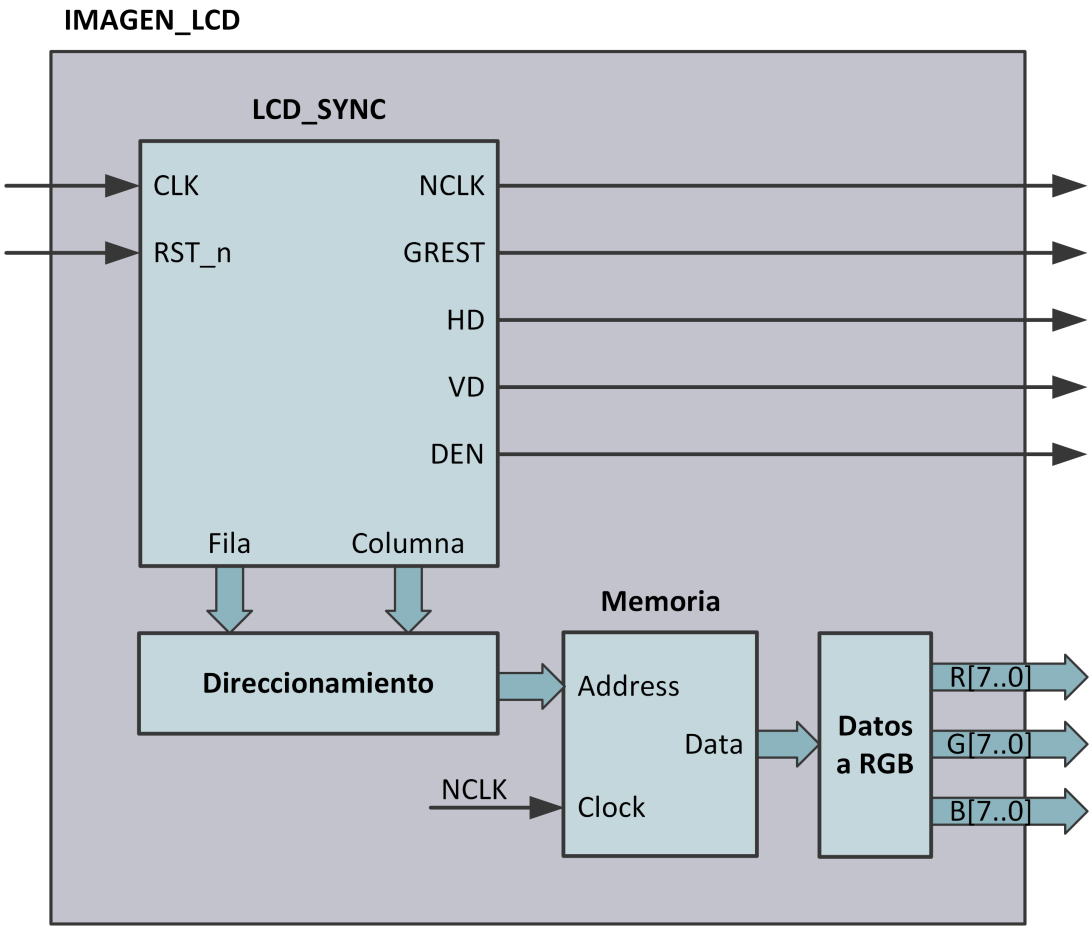
Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA
R[0]	AE25	G[0]	AC22	B[0]	AD15
R[1]	AG22	G[1]	AE22	B[1]	AE16
R[2]	AE24	G[2]	AF21	B[2]	AD21
R[3]	AH22	G[3]	AF22	B[3]	Y16
R[4]	AF26	G[4]	AD22	B[4]	AF15
R[5]	AE20	G[5]	AG25	B[5]	AF24
R[6]	AG23	G[6]	AD25	B[6]	AE21
R[7]	AF20	G[7]	AH25	B[7]	AF25
NCLK	AE15	DEN	AC19	GREST	AH26
HD	AF16	VD	AD19		
CLK	Y2	RST_n	M23		

Subtarea 3: Visualización de una imagen en la pantalla

Sub-task 3: Displaying an image on the screen

En esta subtarea se quieren visualizar en la pantalla una imagen o fotografía previamente almacenada en una memoria ROM interna del dispositivo Cyclone IV. Para ello, partiendo del modulo LCD_SYNC, diseñado y verificado en las subtareas anteriores, habrá que realizar el diseño mostrado en el diagrama de bloques inferior.

In this sub-task, we will display on the screen an image or picture that has been previously stored in an internal ROM memory within the Cyclone IV device. In order to achieve that, we will make use of the LCD_SYNC module from previous steps to build the design shown in the following block diagram.



Tal como se aprecia en el diagrama de bloques, se añadirá al diseño una memoria que contendrá la imagen que se quiere visualizar en la pantalla. Cada posición de memoria tendrá el valor del color de un pixel, por tanto hay que direcciones secuencialmente la memoria, tal como se recorre la pantalla para obtener el color del pixel correspondiente.

El bloque denominado **Direccionamiento** se encargará de generar las direcciones de acceso a las posiciones de memoria en función del valor de la fila y la columna que proporciona el bloque **LCD_SYNC**. El funcionamiento del bloque **Direccionamiento** dependerá del modo de direccionamiento utilizado en el emplazamiento de la imagen en la memoria. Existen dos modos de direccionamiento:

> ***Modo direccionamiento continuo.*** Los pixeles de la imagen se encuentran ubicados en memoria de manera consecutiva, empezando por el pixel del margen superior izquierdo que estará en la dirección cero y siguiendo la secuencia de barrido de la pantalla hacia la derecha hasta completar la primera fila, para a continuación pasar al lado izquierdo de la siguiente fila. Tal como se muestra en la imagen inferior, si se tiene una imagen de ***X* por *Y* píxeles**, hará falta un bus de direcciones de ***k* bits** para recorrer toda la imagen. Necesitando una memoria de al menos ***X* por *Y* posiciones**, para poder contener toda

As can be inferred from the block diagram, a memory will be added to the design that contains the image to be displayed on the screen. Each memory position will contain the color of one pixel. The memory will be addressed sequentially, in much the same way as the screen is scanned, in order to obtain the corresponding pixel color.

The Addressing (**Direccionamiento**) block will be responsible for generating the addresses for memory access depending on the row and column values provided by the **LCD_SYNC** block. Its operation will depend on the addressing mode used to store the image on memory. There are two possible addressing modes:

> **Continuous addressing mode.** Image pixels are stored in the memory sequentially, starting with the pixel at the top left corner, which will be located at address 0, and following the screen scan sequence until the first row is complete, which is then followed by the next row starting from its left end. As outlined in the image below, if we have an image with **X × Y pixels**, an address bus with **k bits** is needed to scan the whole image, and a memory with at least **X × Y positions** is needed to store the whole image. This addressing mode optimizes memory use but, although it may not seem so at first glance, results in a rather complicated address encoding: one must consider the fact that the Row and

la imagen. En este modo de direccionamiento se optimiza el uso de la memoria, pero aunque no lo parezca, se complica el direccionamiento, si se tienen en cuenta que los valores de Fila y Columna tienen en cuenta las zonas "***back porch***" mientras que en memoria se tiene únicamente la imagen tal como se debe visualizar.

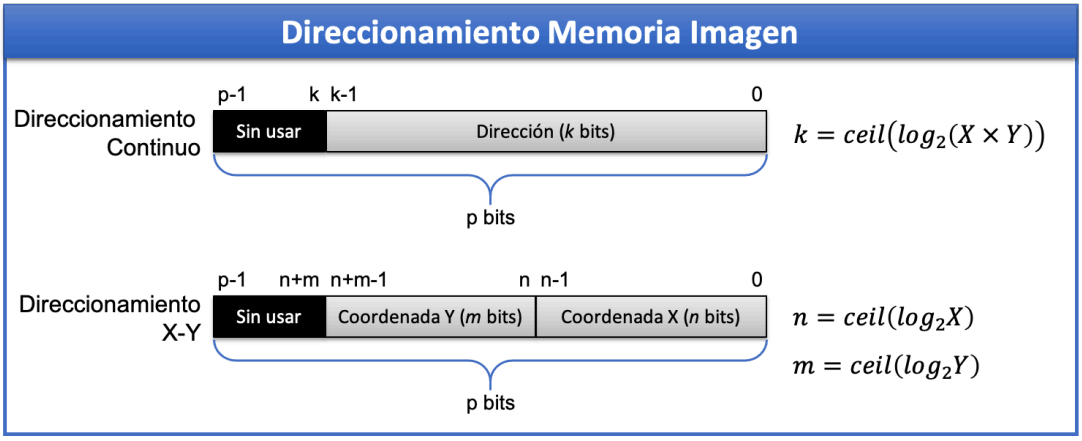
Modo direccionamiento X-Y. En este modo de direccionamiento los pixeles de la imagen se encuentran ubicados en memoria de manera cuasi-consecutiva. Los pixeles correspondientes a una fila de la imagen se encuentran ubicados en la memoria consecutivamente, pero a su vez el primer pixel de una fila siempre comienza en una posición de memoria potencia de dos. Por tanto, para una imagen de ***X* por *Y* píxeles**, la primera fila comienza en la dirección de memoria cero, partir de ahí se encuentran el resto de pixeles de esa fila, mientras que la segunda fila comenzará en la dirección potencia de 2 mayor que ***X***. Por ejemplo, para una imagen de 400 por 240 píxeles, la primera fila comienza en la dirección de memoria cero, la segunda en la dirección 512, la tercera fila en la dirección 1024, etc. De esta forma quedan espacios de la memoria entre las filas de la imagen que no se utilizan, por otra parte la codificación del direccionamiento resulta más sencilla, tal como se

Column values also include the back and front porch areas, while the memory only contains color data for the pixels belonging to the screen display area.

X-Y addressing mode. In this mode, image pixels are stored in the memory in an almost-sequential way. Pixels corresponding to a row are stored consecutively, but the first pixel of any row is always located at an address that is an integer power of two. For an image with **X × Y pixels**, the first row starts at memory address 0 and is stored in the following **X** positions, and the second row starts at the memory position $\mathbf{2^n}$ which is the smallest power of two greater or equal to **X**. For instance, for an image with 400 × 240 pixels, the first row begins at address 0, the second row at address 512, the third one at address 1024, etc. Note that this implies that some parts of the memory, located between the end of one row and the start of the next one, are not used. On the other hand, address encoding is greatly simplified since, as outlined in the image below, the memory address bus value can be obtained by simply concatenating the values of the **X** and **Y** positions. This image also shows how to obtain the necessary memory size for an **X × Y** pixel image: it will be approximately $\mathbf{2^n}$ **× Y** memory positions. As in the previous case, in order to obtain the memory ad-

suita mas sencilla, tal como se muestra en la figura inferior, ya que únicamente consiste en la concatenación del valor de ***X*** e ***Y*** para la obtención del bus de direcciones de la memoria. En la imagen inferior se puede ver el cálculo del numero de bits necesarios, teniendo en cuenta una imagen de ***X*** por ***Y*** píxeles. El tamaño de memoria necesaria, se puede calcular aproximadamente como 2^n ***por*** ***Y***, posiciones. Al igual que antes, la obtención del direccionamiento de memoria a partir de los valores de Fila y Columna, requiere la eliminación de los valores del "***back porch***".

dress from the Row and Column values, one must take into account the existence of the back porch areas.



Otro aspecto a tener en cuenta es la codificación de los colores de las imágenes. Tal como se aprecia en la imagen inferior, existe una gran variedad de opciones a la hora de codificar los colores, desde el blanco y negro representado por un solo bit hasta la codificación de RGB de 30 bits, pasando por la escala de grises de 8 bits. En realidad, esto significa que cada pixel de la imagen que se encuentra en la memoria puede representarse entre 1 bit y 30 bits, por tanto hay que re-codificarlo al formato de RGB 24 bits que utiliza la pantalla, de ahí la necesidad del modulo denominado **Datos a RGB**.

Para realizar la re-codificación del color sirva como sugerencia que si se trata de eliminar bits hay que eliminar los bits de menor peso y si se trata de añadir bits, simplemente ubicamos los bits disponibles como la parte alta y los replicamos en la parte baja de mayor a menos hasta completar. En el caso de pasar de escala de gris a color, hay que poner el mismo valor en los tres colores.

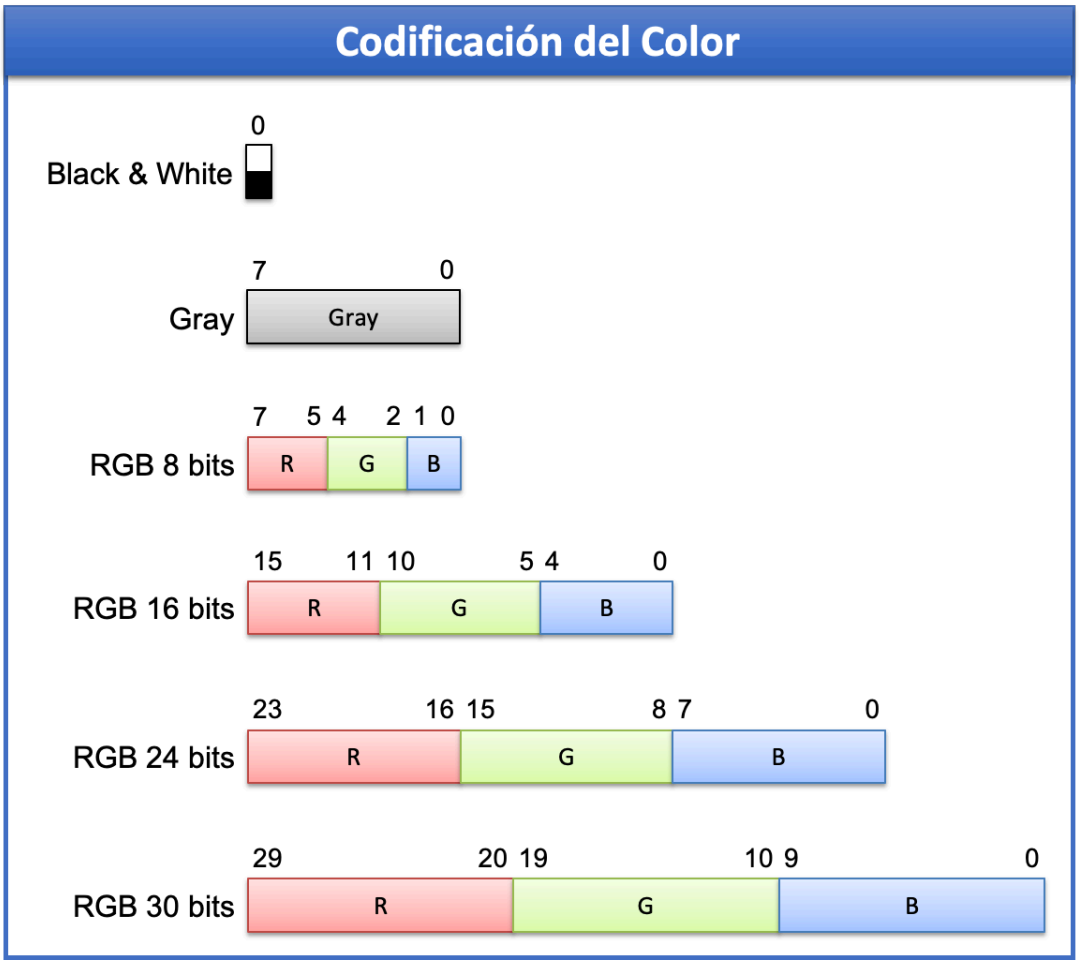
We must also take into account the encoding of image colors. As shown in the image below, there are several standard options for color encoding, from a black-and-white representation where each pixel is represented by a single bit and grayscale representations with 8 bits, up to an RGB encoding using 30 bits (10 per color). This means that each image pixel in the memory could be represented using between 1 and 30 bits, and it might be necessary to re-encode it into the 24-bit RGB format used by our screen. This is where the Data-to-RGB (**Datos a RGB**) block comes into play.

When implementing color re-encoding, follow these guidelines:

► When decreasing the number of bits, always remove the least significant bits.

► When increasing the number of bits, the available bits should be placed in the most significant bit positions and then replicated in the remaining positions.

► When changing from grayscale to an RGB encoding, the same value should be used for all three colors.



Realizar el diseño indicado para visualizar en la pantalla una **imagen de 400 por 240 píxeles, codificada en color RGB de 16 bits, con Direccionamiento X-Y,** que se encuentra en el archivo HEX suministrado a continuación. Hay que tener en cuenta que la pantalla visualiza una **imagen de 800 por 480 píxeles con codificación en RGB de 24 bits**. Por tanto, a parte de la re-codificación del color comentada, a la hora de diseñar el bloque de direccionamiento habrá que tener en cuenta que se deben replicar pixeles de la imagen para aumentar su tamaño al doble.

In order to complete this sub-task, design the proposed module in order to display on screen an **image of 400 × 240 pixels** with **16-bit RGB encoding**, stored in memory with **X-Y addressing**. The image is stored in the HEX file provided below. Remember that our screen displays images with **800 × 480 pixels** and a **24-bit RGB encoding**. Therefore, it will not be enough to implement the color re-encoding described above, but moreover our Addressing block design must be able to replicate image pixels so that its size is doubled.

ARCHIVO IMAGEN EN FORMATO HEX

Archivo en formato HEX con una imagen de 400 por 240 píxeles, codificación de color RGB 16 bits y Direccionamiento X-Y.

Como Crear una memoria ROM

Creating a ROM Memory

La memoria a utilizar en el diseño será una memoria interna del dispositivo. Estas memorias se pueden crear mediante la herramienta **IP Catalog** (en versiones anteriores denominada **MegaWizard Plug-In Manager**) de Quartus II, la cual dispone de un compilador de memorias que permite generar memorias de varios tipos como la ROM, que hace falta en este caso.

Esta memoria guardará la imagen que se quiere visualizar en pantalla. La ROM habrá que inicializarla con la imagen, por medio del archivo que se ha suministrado anteriormente. Este archivo puede ser de tipo HEX (datos en hexadecimal) o MIF (*Memory Initialization File*), y debe guardarse en la carpeta del proyecto.

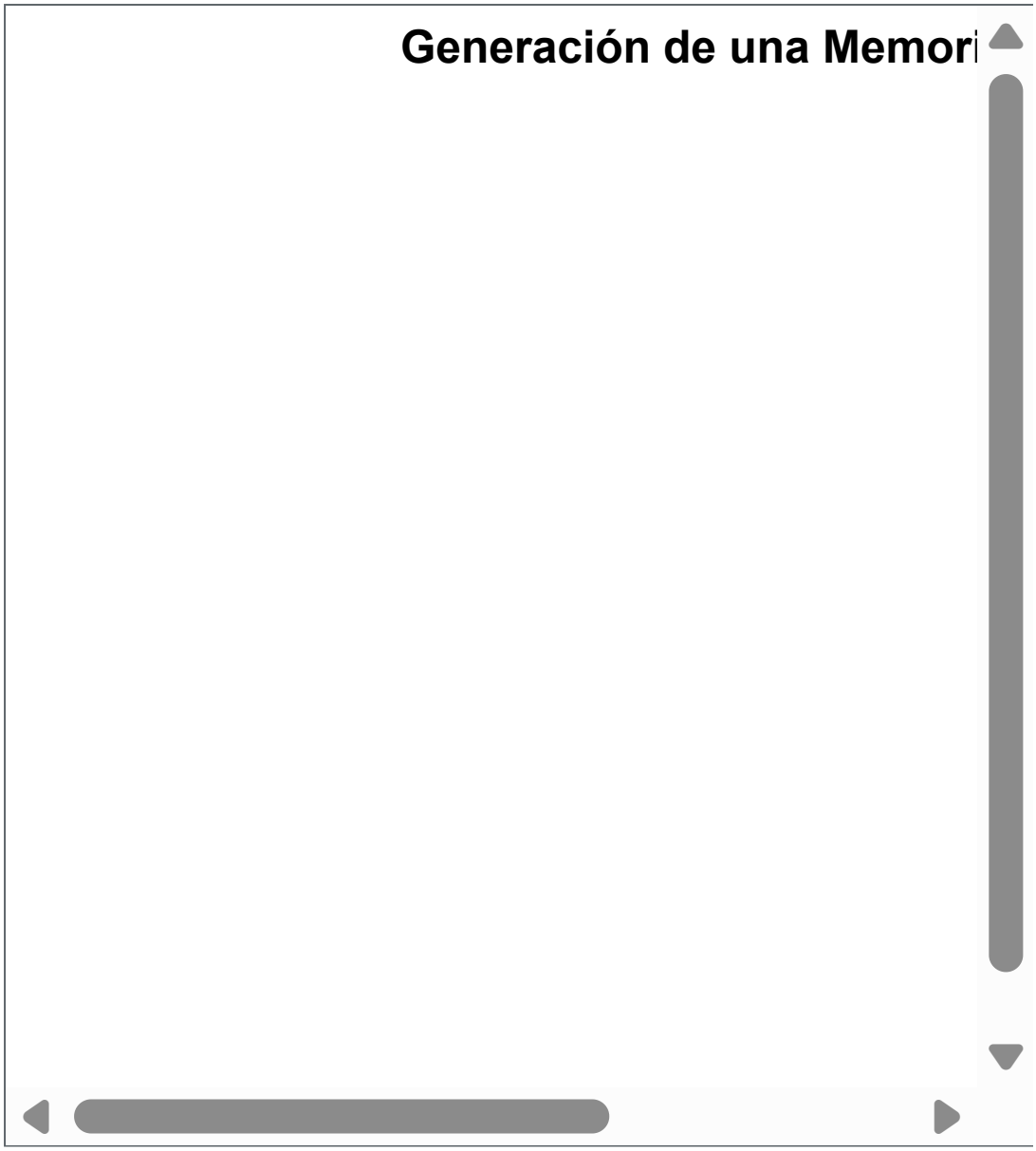
En la Figura inferior denominada **Generación de una memoria ROM**, se puede ver una guía animada de cómo generar una memoria ROM, asi como que opciones y ajustes establecer para instanciar la memoria. El punto de partida para la creación de la memoria ROM, es el proyecto de Quartus donde se realice el diseño. **ATENCIÓN: Los parámetros de anchura de los datos y tamaño de la memoria que se muestran en la animación no son los adecuados para esta aplicación**, sino que son meramente orientativos para ilustrar el uso del IP Catalog. Los valores requeridos para nuestra aplicación son diferentes y deberán escogerse en función de las características de la imagen suministrada.

In this design we will use an internal memory block within the FPGA device. These memory blocks can be created using the **IP Catalog** (previously, "*MegaWizard Plug-In Manager*") tool in Quartus. The tool includes a memory compiler that can generate memories of various sizes and types, including the ROM memory needed for this task.

This memory will be used to store the image to be displayed on the screen. It will be necessary to initialize the ROM memory with the image data using the provided file. This can be either a HEX file (hexadecimal data) or a MIF file (Memory Initialization File), and should be placed in the project folder.

The Figure below, titled **How to generate a ROM memory**, contains an animated guide to the generation of the ROM memory, the options to be selected and the adjustments to be made in order to generate our memory. The starting point for IP Catalog use is our Quartus project where the design is being implemented. **WARNING: The data width and memory size parameters used in the demonstration are not correct for this application**, and only there for demonstration purposes. The values required in our application will be different and must be chosen depending on the image characteristics.

ABRIR EN UNA VENTANA NUEVA



Como resultado de la generación de la ROM, se habrá creado en el directorio del proyecto una serie de archivos, entre los cuales se encuentra el denominado **ROM_image_inst.v**. Este archivo contiene una descripción ejemplo de cómo realizar la instanciación de la memoria ROM generado en el diseño para poderlo utilizar. Su contenido se muestra en la siguiente imagen:

As a result of ROM generation, IP Catalog will create a series of files in our project folder, and in particular one called **ROM_image_inst.v**. This file contains an example description showing how to instantiate the generated ROM in our design. Its contents are shown below:

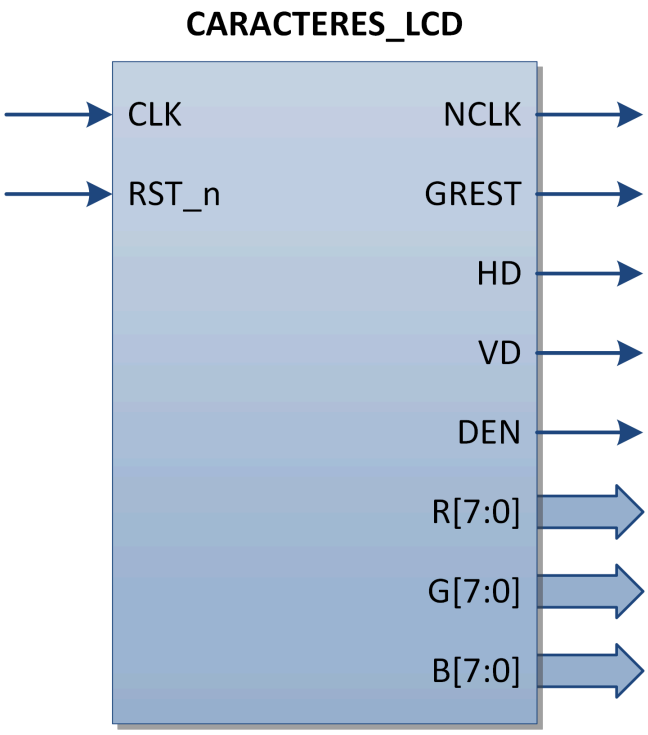
```
ROM_image       ROM_image_inst (
 .address ( address_sig ),
 .clock ( clock_sig ),
 .q ( q_sig )
);
```

Una vez realizado el diseño hay que verificar su funcionamiento mediante simulación con ModelSim y el Emulador de Pantalla utilizado en la sub-tarea anterior. Tras lo cual, se probará su funcionamiento en la placa DE2-115.

Once the module has been designed, verify its operation with a ModelSim simulation and the Screen Simulator used in the previous sub-task. After that, test it on the DE2-115 board.

Subtarea 4: Visualización de caracteres en pantalla

Sub-task 4: Displaying characters on the screen



El objetivo de esta subtarea consiste en visualizar en la pantalla caracteres, concretamente llenar toda la pantalla con caracteres iguales o diferentes. Se ha optado por este tipo de representación porque es mucho más fácil de implementar y resultará más fácil de comprobar el módulo diseñado, ya que no se tiene en cuenta la posición en pantalla sino que solamente hay que gestionar la generación de las señales RGB.

The goal of this sub-task will be to display characters on the screen, more specifically to fill up the screen with equal or different characters. We have chosen this type of representation because it is much easier to implement and to verify the design, as it does not depend on the onscreen position, so one only must take care of the generation of the RGB signals.

Dado que la intención es visualizar en pantalla caracteres (por ejemplo: 0, A, B...), habrá que tener en cuenta, que la pantalla está constantemente actualizándose (refrescándose), por ello es necesario el uso de una memoria para mantener los caracteres y que puedan ser mostrados en pantalla. Para ello se realizará el diseño de un módulo que vaya leyendo de memoria cada carácter y que vaya actuando adecuadamente sobre la señales RGB en función de los valores de Fila y Columna proporcionados por el módulo LCD_SYNC, para sacar el carácter por pantalla.

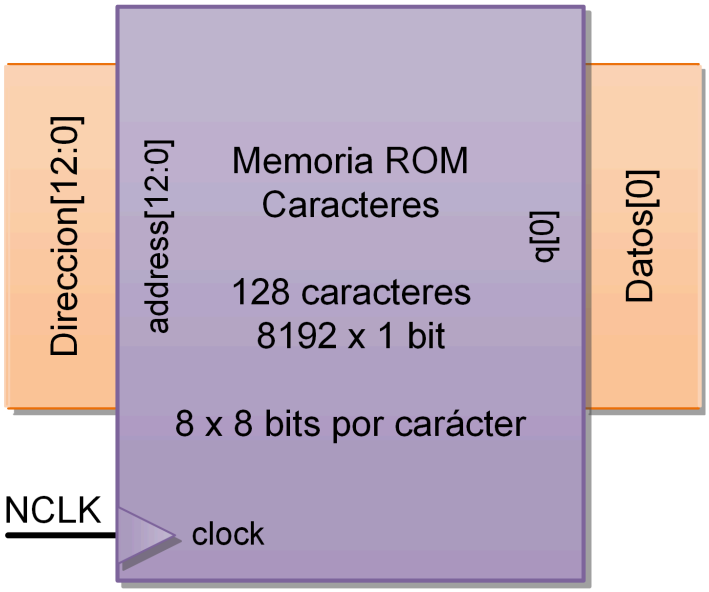
Since our intention is to display characters on the screen (for instance: 0, A, B...), and considering the fact that the screen is constantly refreshing, we will need a memory to keep the character data so they can be represented. The module design to be implemented will read the character data from that memory and then generate the appropriate RGB signals depending on the current Row and Column values provided by LCD_SYNC, so that the character appears onscreen.

La memoria a utilizar en el diseño será una memoria interna del dispositivo o memoria on-chip. Estas memorias se pueden crear mediante la herramienta **IP Catalog** (antes **MegaWizard Plug-In Manager**) de Quartus II, la cual dispone de un compilador de memorias que permite generar memorias de varios tipos como la memoria ROM, que hace falta en este caso.

Una memoria ROM, es un conjunto de celdas donde previamente se han almacenado datos de un tamaño determinado. Para acceder a cada dato simplemente hay que indicar la dirección de la celda que se quiere leer y la memoria nos facilitará el dato almacenado en dicha celda. Por tanto, una memoria ROM consta de un bus de direcciones, cuyo tamaño depende del número de celdas y un bus de datos por medio del cual se proporciona el dato almacenado en una celda. En este caso concreto las memorias de los dispositivos son sincronías, con lo cual necesitan una seña de reloj para funcionar.

In our design, we will use an on-chip memory (internal to the FPGA device) for that purpose. These memory blocks can be created using the **IP Catalog** (previously, *"MegaWizard Plug-In Manager"*) tool in Quartus. The tool includes a memory compiler that can generate memories of various sizes and types, including the ROM memory needed for this task.

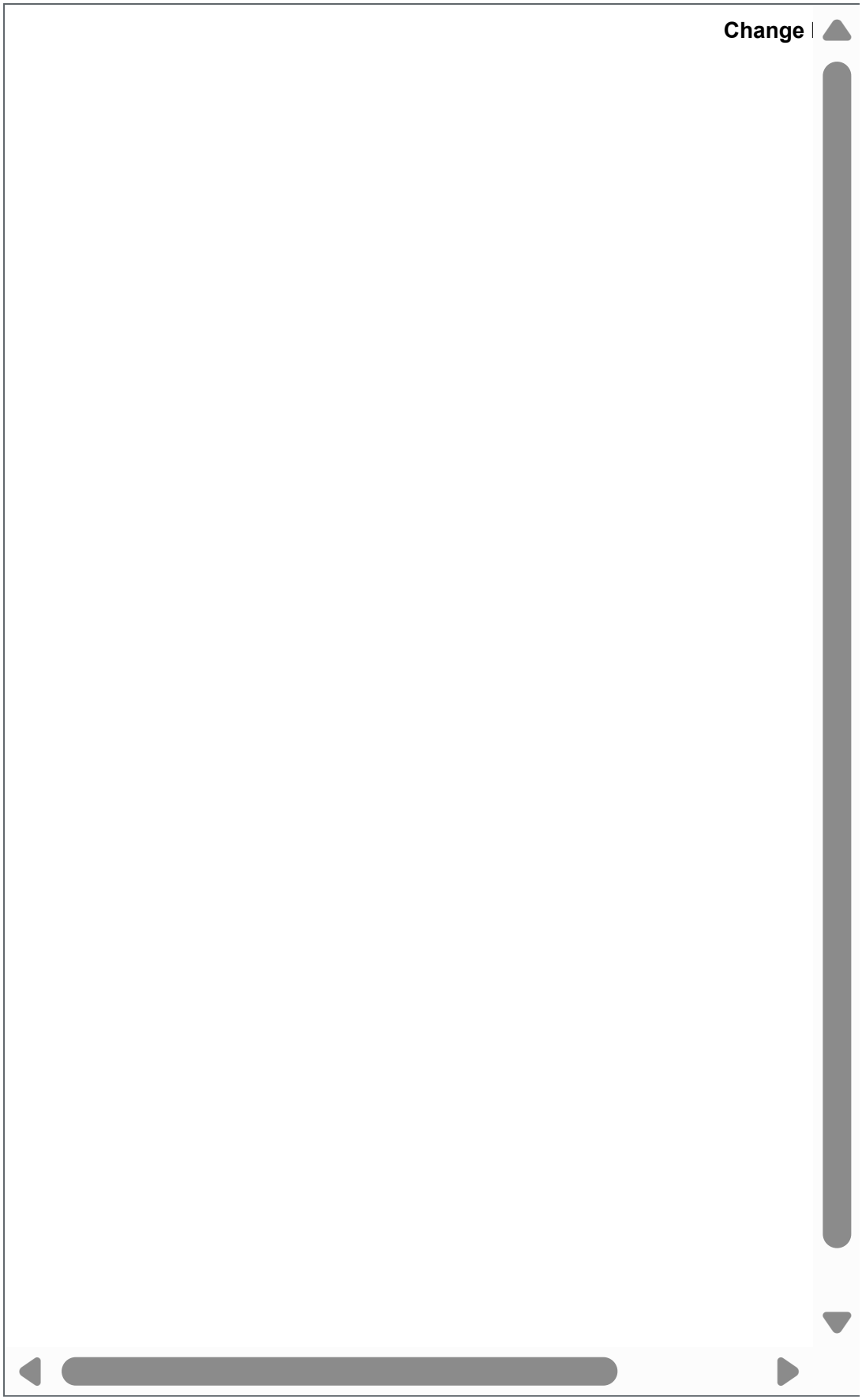
A ROM memory is a set of memory cells where data of a given size have been stored previously. In order to access each data, the address of the cell to be read must be specified, and the memory will output the data stored in that cell. Therefore, the ports of a ROM memory consist of an address bus (input), whose size depends on the number of cells, and a data bus (output), that contains the data read from that cell. In our case, on-chip ROM memories are synchronous, so they also require a clock input.



Esta memoria guardará los caracteres que se quieran visualizar en pantalla. Estos caracteres serán de dimensión 8×8 píxeles. Para cada carácter se indicará mediante un bit, si el pixel correspondiente debe estar a uno o a cero. El contenido de las celdas de la memoria ROM se preconfigura o se inicializa, para medio de un archivo de texto del tipo MIF (Memory Initialization File) o mediante un archivo en formato Intel-HEX, en el cual se establece el valor para cada una de las celdas. El archivo MIF puede definirse usando señales binarias, hexadecimales, decimales u octales. En la siguiente imagen, se muestra la estructura del archivo MIF para la definición de los caracteres de está subtarea. Se trata de una memoria de 8192 celdas con anchura de 1 bits, las direcciones están definidas en decimal y los datos de las celdas en binario, para una mejor comprensión.

This memory will store the characters we wish to display onscreen. These characters will have a size of 8×8 pixels. For each character, one bit will be used to represent each pixel, indicating whether it should be lit or not. The contents of the ROM memory will be initialized with a MIF (Memory Initialization File) text file, that establishes the value for each cell. Such a file can be defined using data with binary, hexadecimal, decimal or octal representation. The next image shows the structure of the MIF file for character definition. It corresponds to a memory consisting of 8192 cells with 1-bit width. Addresses are given in decimal representation and cell data in binary representation so it is easier to read.

ABRIR EN UNA VENTANA NUEVA



El siguiente fichero es un ejemplo del aspecto del archivo de contenido de la memoria en formato MIF, que tiene la definición del aspecto de los caracteres. **¡Advertencia! No utilizar este archivo** en la creación de la memoria, ya que da un error al simular. En su lugar, usar el fichero HEX contenido dentro del apartado *Usando el IP Catalog para Crear una ROM* más abajo.

The following file shows an example of how a MIF memory content file would look, containing the definition of all characters. **WARNING: Do not use this file** when creating the ROM memory, as it will not be simulated correctly by ModelSim. In its place, use the HEX file that is provided in the *Using IP Catalog to create a ROM* section below.

ARCHIVO DE CONTENIDO DE MEMORIA EN FORMATO MIF: CHAR_ROM.MIF

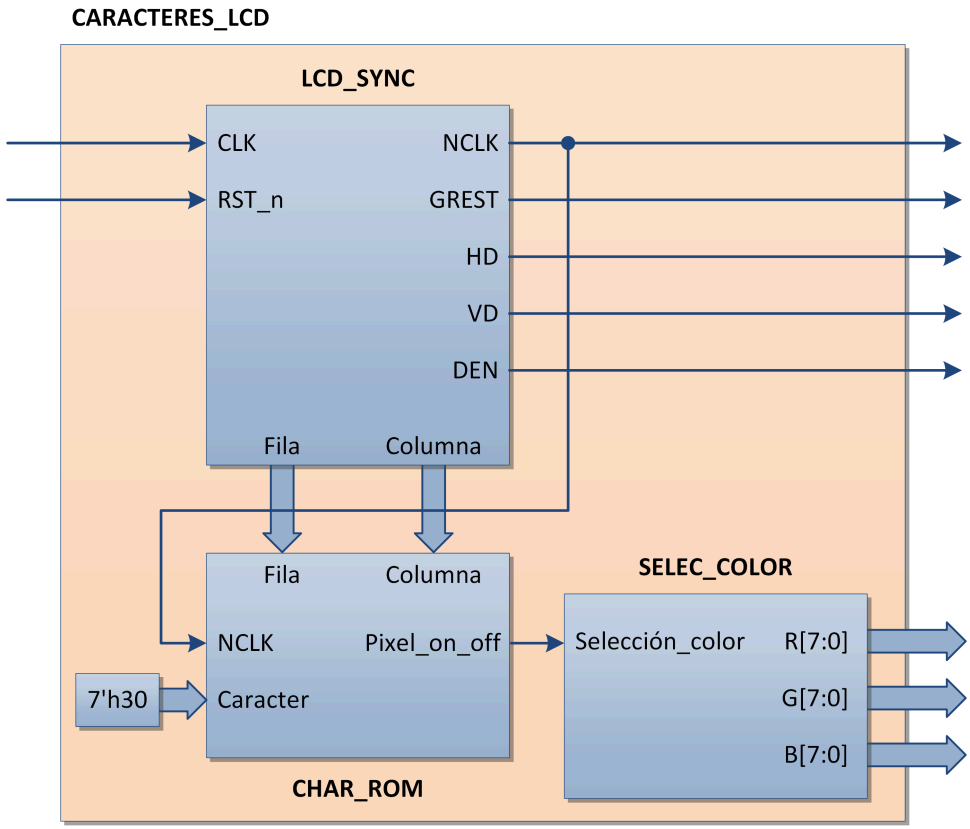
Ejemplo del aspecto del archivo de contenido de la memoria en formato MIF, que tiene la definición del aspecto de los caracteres. ¡Advertencia! No utilizar este archivo en la creación de la memoria, da un error al simular.

Propuesta Estructura Diseño

Proposed Design Structure

A partir de lo explicado anteriormente, se puede plantear el diagrama de bloques de la imagen como propuesta orientativa para realizar el diseño. Este constará básicamente de tres bloques: el **LCD_SYNC**, el **SELEC_COLOR** y el **CHAR_ROM**.

Based on the previous exposition, we propose the design structure outlined in the block diagram in the image below. The design will consist in three blocks: **LCD_SYNC**, **SELEC_COLOR** and **CHAR_ROM**.

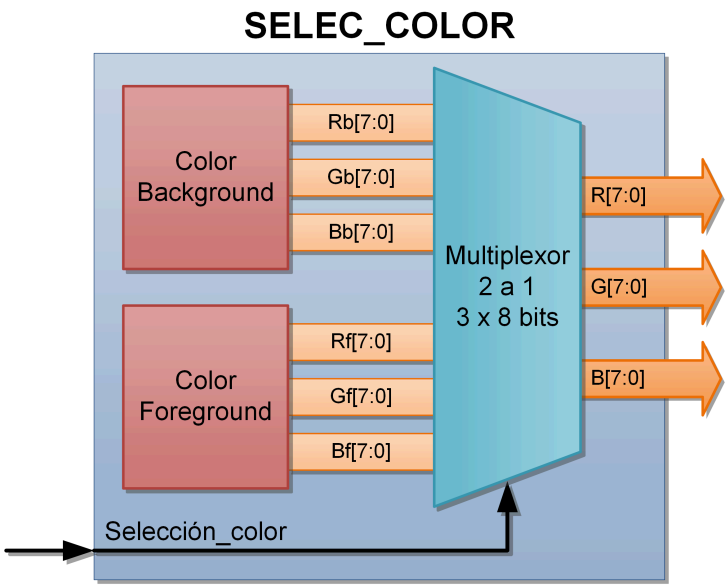


El bloque **LCD_SYNC**, corresponde con el bloque diseñado en la primera subtarea y que es el encargado de generar las señales de sincronismo de la pantalla.

El bloque **SELEC_COLOR**, es el encargado de generar los valores de las señales RGB, según el color que se desee. Consiste en un simple multiplexor que permite seleccionar entre el color estipulado para representar el fondo de la pantalla o background y el color el utilizado para representar los caracteres en la pantalla o foreground. La selección se realiza a partir de una señal proveniente del **CHAR_ROM** que indica si un pixel corresponde con el fondo o con el carácter.

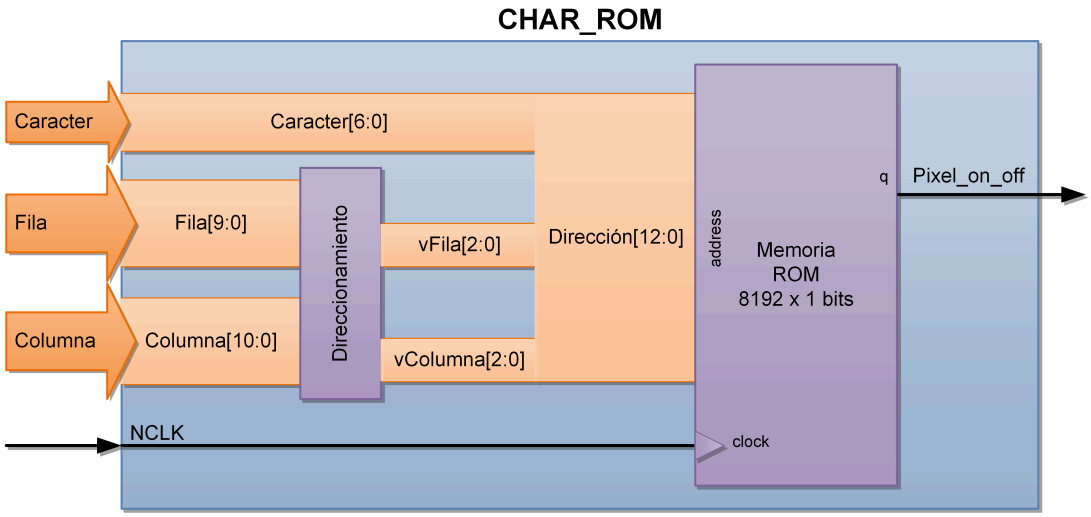
The **LCD_SYNC** block has already been designed in the first sub-task, and is in charge of generating the synchronism signals needed by the screen.

The **SELEC_COLOR** block will generate the values of the RGB signals according to the desired display colors. It consists in a multiplexer that chooses between a chosen background color and a chosen foreground color used for character representation. Selection between both colors is based on a signal from the **CHAR_ROM** block that determines whether a given pixel should be lit (foreground color, corresponds to the character) or not (background color, corresponds to the screen background).



El bloque **CHAR_ROM**, fundamentalmente está compuesto por la memoria ROM explicada anteriormente, y un bloque de **Direccionamiento**, que se interconectará tal como se muestra en el diagrama de bloques inferior. El bloque de **Direccionamiento** tendrá la misma funcionalidad que el utilizado en la Sub-Tarea anterior, generando a partir de los valores de Fila y Columna (indicados por el **LCD_SYNC**) las direcciones de la memoria. El bus de direcciones de la memoria, se complementará mediante la concatenación en la parte alta de la posición que ocupa el caracter que se quiere visualizar, normalmente su posición concuerda con el valor ASCII del caracter. Así, la memoria dará a la salida un dato de 1 bit correspondiente al pixel de una fila y una columna del carácter. Obteniéndose de este modo, una salida que indicará si un pixel concreto debe representar el fondo (0) o el caracter (1).

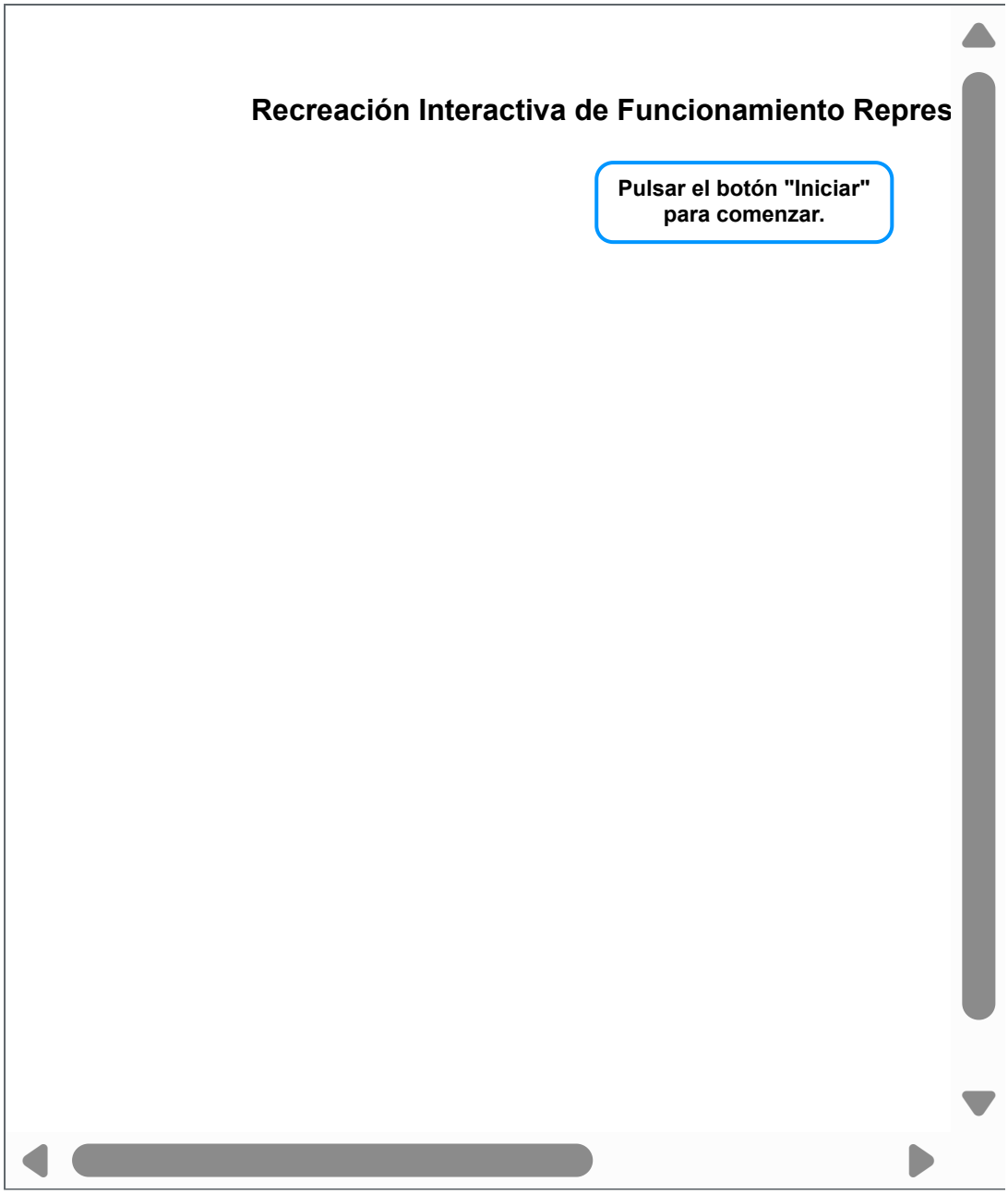
The **CHAR_ROM** block is made up of the ROM memory discussed above and an Addressing block ("**Direccionamiento**") that will be connected as shown in the block diagram below. The Addressing block will work in the same way as in the previous sub-task, converting the Row and Column values given by **LCD_SYNC** into valid row and column values corresponding to the character to be displayed. The memory address bus will contain the 3-bit column and row plus the number of the character to be displayed, according to its order within the character ROM memory. In this case, this order agrees with the ASCII code of each character. The memory will thus output a 1-bit value that corresponds to the pixel in a particular row and column for a particular character. This bit output determines whether a particular pixel should represent the background (0) or the character (1).



A continuación se muestra una animación interactiva en la que se ilustrar el funcionamiento conjunto de los bloques CHAR_ROM y SELEC_COLOR.

The following interactive animation is meant to demonstrate the joint operation of the **CHAR_ROM** and **SELEC_COLOR** blocks.

ABRIR EN UNA VENTANA NUEVA



Usando el IP Catalog para Crear una ROM

Using IP Catalog to create a ROM

La memoria a utilizar en el diseño será una memoria interna del dispositivo. Estas memorias se pueden crear mediante la herramienta **IP Catalog** (en versiones anteriores denominada **MegaWizard Plug-In Manager**) de Quartus II, la cual dispone de un compilador de memorias que permite generar memorias de varios tipos como la ROM, que hace falta en este caso.

Esta memoria guardará los caracteres que se quieran visualizar en pantalla. Estos caracteres serán de dimensión 8 x 8. La ROM habrá que inicializarla con estos caracteres, para ello se utilizará el archivo tipo HEX, que se suministra. Este archivo puede definirse usando señales binarias, hexadecimales, decimales u octales. El archivo que se suministra define una memoria de 8192 celdas con anchura de 1 bit. Las direcciones son definidas en decimal y los datos de las celdas en binario, para una mejor comprensión.

In this design we will use an internal memory block within the FPGA device. These memory blocks can be created using the **IP Catalog** (previously, "*MegaWizard Plug-In Manager*") tool in Quartus. The tool includes a memory compiler that can generate memories of various sizes and types, including the ROM memory needed for this task.

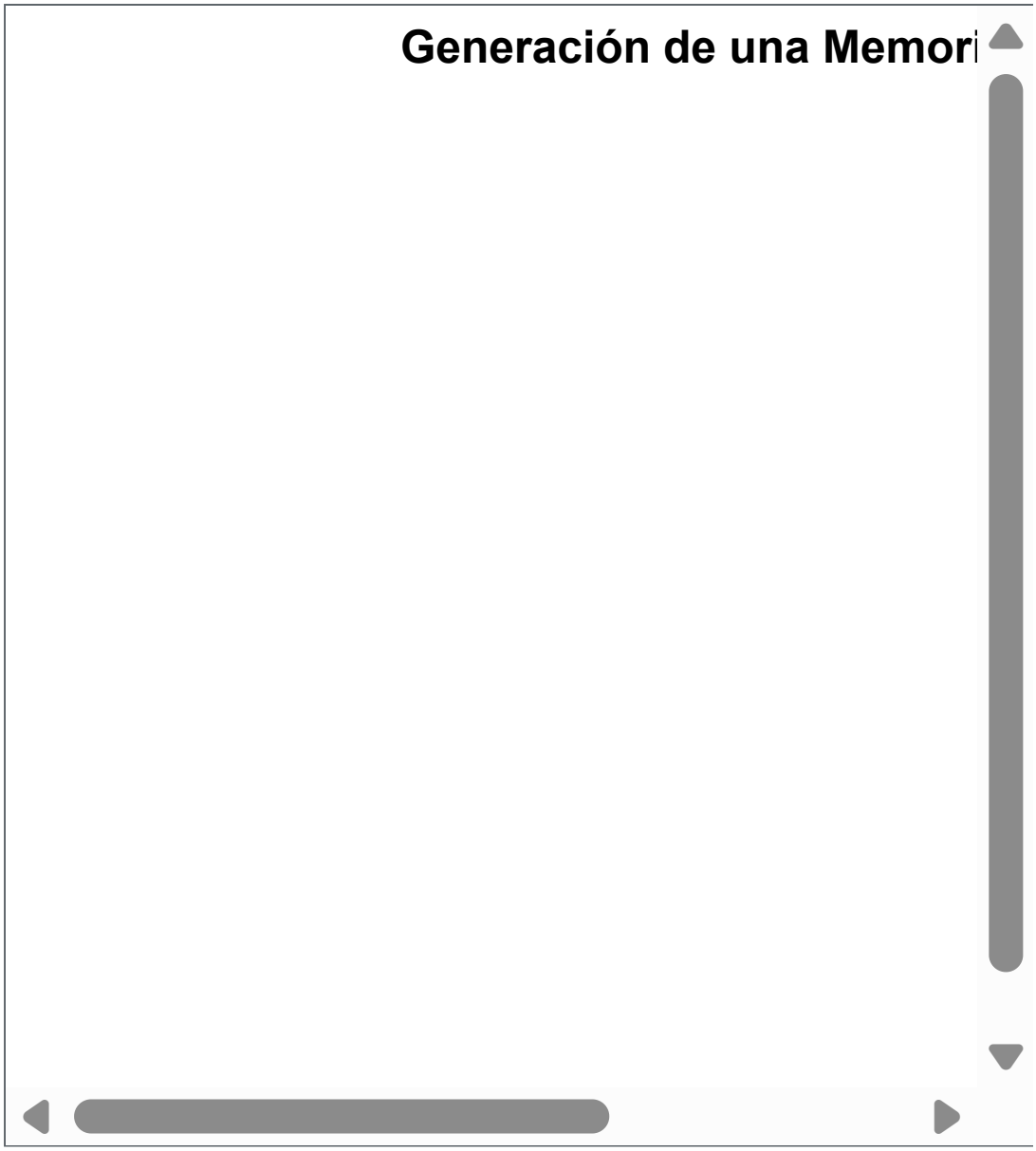
This memory will store the character data to be displayed onscreen. Characters will have a size of 8 × 8 pixels. We will need to initialize the ROM with these character data, using the provided HEX file. Its contents can be specified using binary, hexadecimal, decimal or octal data. The file we provide defines a memory with 8192 cells with 1-bit width. Addresses are given in decimal and cell data in binary representation.

ARCHIVO DE CONTENIDO DE LA MEMORIA: CHAR_ROM.HEX

En la Figura inferior denominada **Generación de una memoria ROM**, se puede ver una guía animada de cómo generar una memoria ROM, que parámetros hay que indicar y que opciones y ajustes establecer para instanciar la memoria necesaria para el diseño. El punto de partida para la utilización del **IP Catalog**, es el proyecto de Quartus donde se realice el diseño.

The Figure below, titled **How to generate a ROM memory**, contains an animated guide to the generation of the ROM memory, the options to be selected and the adjustments to be made in order to generate our memory. The starting point for **IP Catalog** use is our Quartus project where the design is being implemented.

ABRIR EN UNA VENTANA NUEVA



Como resultado de la generación de la ROM, se habrá creado en el directorio del proyecto una serie de archivos, entre los cuales se encuentra el denominado **ROM_char_inst.v**. Este archivo contiene una descripción ejemplo de cómo realizar la instanciación de la memoria ROM generado en el diseño para poderlo utilizar. Su contenido se muestra en la siguiente imagen:

As a result of ROM generation, IP Catalog will create a series of files in our project folder, and in particular one called **ROM_char_inst.v**. This file contains an example description showing how to instantiate the generated ROM in our design. Its contents are shown below:

```
ROM_char        ROM_char_inst (
  .address ( address_sig ),
  .clock ( clock_sig ),
  .q ( q_sig )
);
```

Mejora Visual del Diseño

Al probar el diseño en de esta subtarea en el hardware, se observa claramente que las letras son muy pequeñas, ya que la pantalla tiene una capacidad de 100 por 60 caracteres, lo cual es excesivo para tan reducido espacio de visualización.

Una mejora del diseño es incrementar el tamaño de los caracteres a visualizar, lo cual se puede realizar de manera sencilla replicando los pixeles de los caracteres tanto en horizontal como en vertical. Esta es una modificación sencilla que únicamente requiere de modificar dos lineas en el bloque de **Direccionamiento** del diseño anterior.

Realizar la modificaciones oportunas para que los caracteres tenga un tamaño el doble, es decir sean de 16 por 16 pixeles, viéndose mejor así en la pantalla.

Visual Improvement of the Design

When testing this sub-task's design on hardware, the resulting characters are very small, as the screen can represent up to 100 × 60 characters of the given size. This is excessive, given the small size of the display area.

One possible improvement to the design consists in increasing the size of the displayed characters. This can be achieved easily by replicating the character pixels both horizontally and vertically. This easy modification only requires changing a couple of code lines in the previous design.

Apply these modifications to your code so that characters appear with double size (16 × 16 pixels each) on the screen, making them easier to be read.

Subtarea 5: Generación de una frase en la pantalla

En esta subtarea se quiere visualizar los nombres de los componentes del grupo de prácticas en la pantalla, en una línea bastante centrada. Como punto de partida se puede tomar el diseño realizado en la subtarea anterior, con la diferencia de que la elección de carácter a visualizar no es un valor fijo, sino que va a ser unas direcciones cambiantes según el posicionado en pantalla, controlado de nuevo por los contadores de filas y columnas.

Una sugerencia importante sería almacenar en otra memoria ROM la frase a visualizar, concretamente el contenido de esa memoria serían las direcciones a seleccionar en la memoria CHAR_ROM y se direccionaría esa nueva memoria mediante los valores de las filas y de las columnas que se tiene accesibles. El correcto direccionamiento y la correcta generación del archivo tipo MIF de inicialización, marcarán el éxito o no que se pueda tener en este ejercicio.

Sub-task 5: Displaying a message on the screen

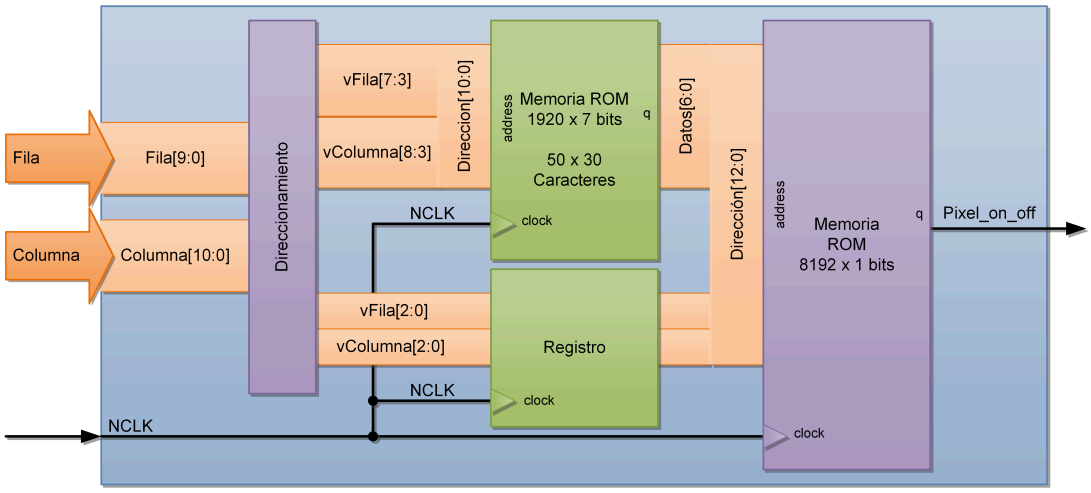
The goal in this sub-task is to display the names of the members of your lab group on the screen, in a line as centered as possible. Use the design from the previous sub-task as a starting point. The main difference is that the character to display is not a fixed value now, but rather a changing value depending on the screen position, determined by the row and column counters.

An important suggestion is to store the sentence to be displayed in another ROM memory. This new memory would be addressed using the row and column counter values, and the contents of the ROM would be the character addresses to select on the CHAR_ROM memory. Success or failure in this exercise will be determined by correct addressing and correct generation of the MIF file for initialization.

La mejor opción es implementar una memoria de pantalla para el texto. Se trataría de utilizar la misma filosofía que en la subtarea de visualización de la imagen, pero aplicada a caracteres en lugar de píxeles. A partir de la subtarea anterior, se añadiría una memoria que proporcione el valor del carácter que se quiere visualizar en cada instante. En el diagrama de bloques inferior se puede ver una propuesta de diseño.

The best option is to implement a screen memory for the text. This design uses the same philosophy as in the sub-task for image display, but applied to characters instead of image pixels. Starting from the previous subtask, we add a memory block that provides the value of the character to be displayed at each moment. A design proposal can be seen in the block diagram below.

TEXTO


Fila
Fila[9:0]
Columna
Columna[10:0]
Direccionamiento
vFila[7:3]
vColumna[8:3]
Direccion[10:0]
address
Memoria ROM
1920 x 7 bits
50 x 30
Caracteres
q
Datos[6:0]
NCLK
clock
vFila[2:0]
vColumna[2:0]
Registro
NCLK
clock
NCLK
Dirección[12:0]
address
Memoria
ROM
8192 x 1 bits
q
Pixel_on_off
clock

En este diagrama se parte de la versión de la subtarea anterior en la que se muestran los caracteres con un tamaño de 16 × 16 pixeles. Por tanto, en la pantalla caben 50 × 30 caracteres. Eso significa que es necesario añadir una memoria (mostrada en color verde en el diagrama de bloques) que sea capaz de albergar dicha cantidad de caracteres, de tal forma que cada posición de la memoria contenga el valor del carácter que se quiera visualizar. Para simplificar el direccionamiento de la memoria, al igual que se hizo en la visualización de una imagen, se empleará un **direccionamiento del tipo X-Y**, cuyo valor se obtendrá del dato de Columna y Fila por medio del bloque de Direccionamiento. La posición que ocupa un carácter vendrá dada mediante una coordenada horizontal que variará entre 0 y 49 y una coordenada vertical que variará entre 0 y 29.

El direccionamiento X-Y conlleva que la memoria de texto deba tener un tamaño de 1920 palabras de 7 bits. Se utilizará una memoria ROM que contendrá el mensaje que se quiere visualizar. Por tanto, hay que crear un archivo de inicialización de la memoria en **formato MIF** que contenga los códigos de los caracteres a visualizar en las posiciones adecuadas. Hay que destacar que esta memoria está continuamente recorriéndose, por tanto, deben inicializarse todas las posiciones de la memoria con un valor correspondiente al código del espacio en blanco, que en esta ocasión es el 0 o también el 0x20.

A continuación se incluye un ejemplo

This diagram starts from the version of the previous subtask in which characters with a size of 16 × 16 pixels are displayed. So there is room for 50 × 30 characters on the screen. This means that it is necessary to add a memory block (shown in green in the block diagram) that is capable of holding said quantity of characters, in such a way that each memory position contains the value of the character to be displayed. To simplify memory addressing, an **X-Y type addressing** will be used as was done in the image display subtask. The memory address value will be obtained from the Column and Row data by means of the Addressing block. The position that a character occupies will be given by a horizontal coordinate that will vary between 0 and 49 and a vertical coordinate that will vary between 0 and 29.

The use of X-Y addressing implies that the text memory must have a size of 1920 7-bit words. A ROM memory will be used that will contain the message to be displayed. Therefore, it is necessary to create a memory initialization file **in MIF format** that contains the codes of the characters to be displayed at their adequate positions. It should be noted that this memory is continuously being scanned, therefore all memory positions must be initialized with a value corresponding to the blank space code, which on this occasion is either 0 or also 0x20.

The following is an example of a MIF file for the text memory, containing the names of two students on two dif-

de fichero MIF para la memoria de texto que contiene los nombres de dos alumnos en dos líneas separadas. Puede usarse como punto de partida para el diseño.

ferent lines. It can be used as a starting point for your own design.

TEXT_ROM_EJEMPLO.MIF
Ejemplo de fichero MIF con el contenido de la memoria de texto.

Una vez verificado el diseño en la placa, se puede hacer una modificación dinámica del texto mostrado en la pantalla por medio de la herramienta de Quartus denominada "*In-System Memory Content Editor*", que permite el acceso a la memoria ROM para la modificación de su contenido en tiempo real. Abajo se muestra un pequeño video de uso de la herramienta. Utilizar esta herramienta para modificar el contenido de la memoria de texto con el diseño en marcha y comprobar que el texto se actualiza en la pantalla LCD. Incluir capturas de pantalla y/o fotos de la prueba en la memoria del trabajo.

Once the design has been verified on the evaluation board, it is possible to test dynamic modification of the text displayed on the screen by means of the "*In-System Memory Content Editor*" tool in Quartus, which allows access to the ROM memory for the modification of its content in real time. Use this tool to modify the contents of the text memoria while the design is running and check that the text is updated automatically on the LCD screen. Include screenshots and/or pictures of the experiment in the submitted report.

Subtarea 6: Visualización de una imagen y de texto en la pantalla

Sub-task 6: Displaying an image and a message on the screen

En esta subtarea se pretende unir la visualización de una imagen en la pantalla realizada en la subtarea 3 con la escritura de texto encima de la imagen realizada en la subtrarea anterior. En la figura inferior se aprecia como se ha escrito el texto "Palau de les Arts" encima de la imagen.

In this subtask the aim is to unite the display of an image on the screen carried out in subtask 3 with the writing of text on top of the image carried out in the previous subtask. The figure below shows how the text "Palau de les Arts" has been written over the image.

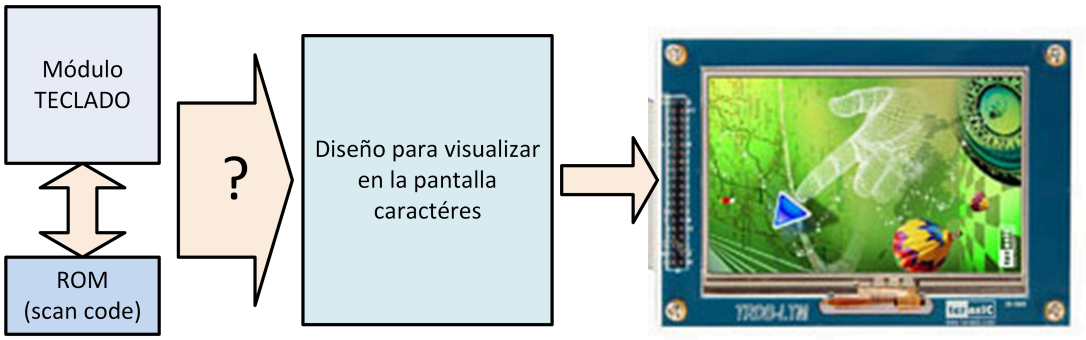
Subtarea Opcional

Subtarea 7: Escribir desde el teclado y visualizar en pantalla

Tras la realización de las subtareas anteriores en las que se controlaba la visualización en la pantalla y en la que se representaban caracteres en una posición determinada, se quiere implementar un módulo que se utilice para visualizar en la pantalla las teclas pulsadas en un teclado PS/2 conectado a la placa DE2-115, tal y como se muestra en la siguiente imagen:

Sub-task 7: Typing on keyboard, displaying onscreen

After the previous sub-tasks, dedicated to controlling onscreen display of images and representation of characters at a given position, we wish to implement a design capable of displaying on the screen the keys pressed on a PS/2 keyboard connected to the DE2-115 board, as outlined in the following image:



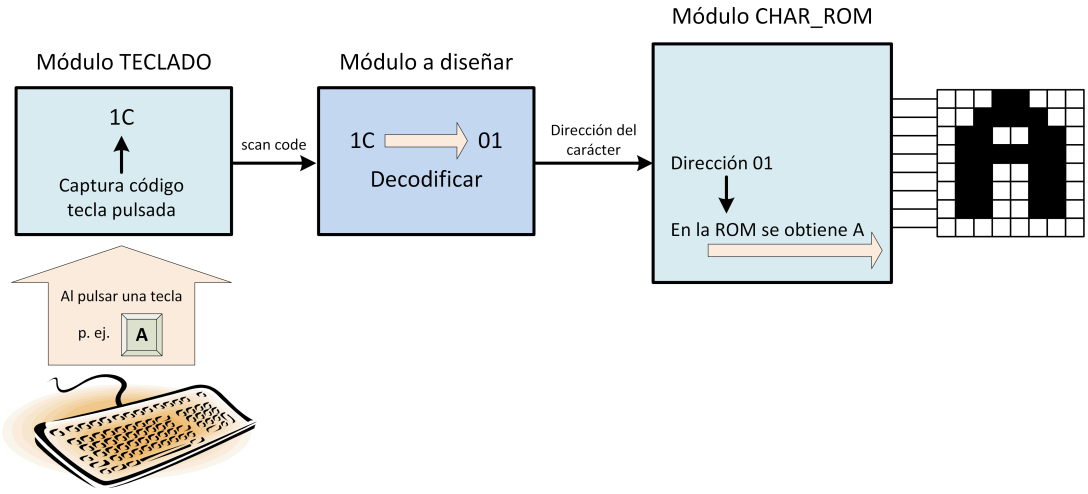
Para que no haya que realizar el diseño de control del teclado, a continuación se encuentra disponible una versión de dicho bloque, realizada por los profesores de la asignatura. El módulo de control de teclado facilitado proporciona el valor de la tecla pulsada en cada momento. Dicho dato es lo que se conoce como el "scan code" de la tecla, el cual es único y universal para todos los teclados.

It is not necessary to design the keyboard controller. Instead, we provide next an implementation of this design, made by the course's instructors. The keyboard controller module provides an identifier of the key that is being pressed at any given moment. This identifier is known as the key's "scan code", and is a standard value, universal for all keyboards.

PROYECTO DE QUARTUS DE EJEMPLO DE USO DE TECLADO: VISUALIZA.QAR

El "scan code" proporcionado por el módulo del teclado, deberá ser transformado en un código que permita actual sobre la ROM de texto, para seleccionar el carácter correspondiente con la tecla pulsada y representada mediante dicho "scan code". Por tanto habrá que realizar el diseño de un módulo que se encarga de dicha descodificación, tan como se muestra en la siguiente imagen:

The scan code provided by the keyboard module must be transformed into a different code that can be used as an input to the text ROM, so that the correct character is selected for each scan code. Therefore, a module must be designed that implements this decoding operation, as shown in the following image:



Por ejemplo: si se pulsa la tecla 'A' del teclado, el código que se transmite por la línea o "scan code" es '0x1C'. Cuando se reciba este código el módulo a implementar debe actuar de decodificador y pasar este código al código correspondiente a la letra A, que está contenido en la archivo MIF del módulo CHAR_ROM, es decir se traducirá por la dirección 0x41.

For example: if the 'A' key is pressed on the keyboard, the scan code transmitted to the FPGA and output by the keyboard controller module is '0x1C'. When this code is received, the module to be designed must decode it and transform it into the code corresponding to the character 'A', i.e. the position where it is contained in the CHAR_ROM MIF file, which is address 0x41.

Esta tarea puede ser abordada partiendo del diseño realizado por la subtarea anterior que mostraba un texto en la pantalla. Sobre dicha tarea hay que realizar una modificación consistente en cambiar la memoria ROM que contiene el texto, por una memoria RAM de doble puerto, de tal forma que haya un puerto de lectura que funcione igual que en la tarea anterior y haya otro puerto de escritura donde se escriben los caracteres provenientes del teclado.

This task can be approached starting from the design carried out by the previous subtask that showed a text on the screen. On this task, a modification must be made consisting of changing the ROM memory that contains the text, for a dual-port RAM memory, in such a way that there is a reading port that works the same as in the previous task and there is another writing port where the characters coming from the keyboard are written.

El teclado se conecta a la FPGA por medio de dos puertos adicionales por uno de ellos circulan los datos y por el otro la señal de sincronización o reloj para la captura de los datos, ya que se trata de una comunicación serie. En la tabla inferior se muestra la asignación de pines correspondiente a las dos señales del teclado.

The PS/2 keyboard is connected to the FPGA using two additional ports that implement a serial communication interface. One of these ports is used for data, and the other one for a synchronization or clock signal used to time data capture. The table below contains the necessary pin assignments for these two keyboard signals.

Nombre Puerto	Pin FPGA	Nombre Puerto	Pin FPGA
PS2_CLK	G6	PS2_DAT	H5